

Subgraph Enumeration: Beyond Tree Decomposition [Technical Report]

Qiyang Li

The Chinese University of Hong Kong
Hong Kong, China
qyli@se.cuhk.edu.hk

Jeffrey Xu Yu

The Hong Kong University of Science
and Technology (Guangzhou), China
jeffrexyu@hkust-gz.edu.cn

Zongyan He

The Chinese University of Hong Kong
Hong Kong, China
zyhe@se.cuhk.edu.hk

ABSTRACT

We address the subgraph enumeration problem: given an unlabeled pattern graph p and an unlabeled data graph G , find all subgraphs in G isomorphic to p . Unlike labeled matching, the absence of label constraints creates exponentially larger search spaces with limited pruning opportunities. To address this challenge, we follow tree decomposition (TD) approaches that break complex patterns into smaller subgraphs (bags), compute matches for each bag, and join them to obtain final results. However, existing TD approaches suffer from suboptimal decomposition selection, incomplete symmetry-breaking usage, and expensive intermediate result materialization. We present MDSE (Minimal Decomposition-based Subgraph Enumeration) with three key contributions. We introduce minimal fractional hypertree decompositions (MinFHDs) that ensure compact bags and an efficient algorithm to explore all optimal-width decompositions. We develop new symmetry-breaking integration using complete rule sets with systematic selection for maximum pruning effect. To reduce materialization costs, we design MixJoin by embedding final result assembly within bag processing and formulate an enhanced cost model for attribute orders, incorporating both intersection and materialization overhead. Evaluation across 101 pattern graphs and 8 real-world datasets shows MDSE substantially outperforms existing algorithms.

PVLDB Reference Format:

Qiyang Li, Jeffrey Xu Yu, and Zongyan He. Subgraph Enumeration: Beyond Tree Decomposition [Technical Report]. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/magic62442/subgraph-enumeration>.

1 INTRODUCTION

Given an unlabeled pattern graph p and an unlabeled data graph G , subgraph enumeration finds all subgraphs in G that are isomorphic to p . As one of the fundamental graph analysis operations, subgraph enumeration is widely used in real-world applications, including motif discovery [69], fraud detection [51], graph comparison [55], and functional prediction [7]. Unlabeled subgraph enumeration naturally arises in quantum computing compilation,

where a compiler must map virtual qubits to physically connected qubits on quantum hardware. Since qubits are homogeneous and differ only in connectivity, this mapping is solved as an unlabeled subgraph matching problem [59], where multiple mappings must be evaluated since different mappings lead to different costs.

Traditional approaches for subgraph enumeration rely on backtracking algorithms [65] or worst-case optimal join algorithms (WCOJ) [46, 48, 66]. Sun et al. [62] show that these two algorithms are essentially equivalent. When node labels are available, these algorithms can be enhanced through sophisticated filtering and pruning techniques [3, 5, 6, 14, 23, 24, 33, 34, 43, 54, 62, 79]. The surveys summarizing these methods can be found in [30, 61, 77].

However, these labeled matching techniques cannot be effectively applied to unlabeled graphs due to the lack of label constraints. For unlabeled subgraph enumeration, researchers have developed various structural techniques to improve efficiency. These include symmetry-breaking methods that exploit automorphisms to eliminate redundant enumerations [19], intermediate result reuse strategies that cache neighbor set intersections across backtracking branches [21, 22, 60], and independent set optimizations that avoid unnecessary intersections when matching independent nodes in the pattern graph p [56, 57, 60]. Additionally, parallel and distributed approaches have been explored to scale across multiple cores and machines [35–37, 50, 52, 60, 72]. However, the performance improvements from these structural techniques remain limited for complex patterns, as the fundamental exponential search space is not substantially reduced.

To further reduce the search space, decomposition-based approaches have emerged as an alternative strategy. These methods partition a complex pattern graph into multiple sub-queries or join units $\{p_i\}$ and assemble their results to obtain matches for the original pattern p . Early decomposition approaches used join units such as stars [36, 63], cliques [37], and path sketches [76] that are subgraphs induced by selected paths of p . However, these approaches face a fundamental dilemma: either join unit processing is complex, or assembly is complex. For cliques and stars, join unit processing is efficient since their sizes are bounded, but assembly becomes expensive due to complex cyclic joins. Conversely, path sketches enable simple assembly but may result in large subgraphs as join units, making unit processing complex.

Tree decomposition (TD) [17] provides a more principled approach that does not restrict join units to a specific type. Given a pattern p , tree decomposition constructs a tree-structured representation T where each tree node (termed a bag) corresponds to a subgraph of p . The algorithm computes subgraph matches for individual bags, materializes intermediate results as relations, and joins these relations to produce final matches. Tree decomposition offers

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

key advantages: it bounds bag sizes by the tree width or fractional hypertree width while guarantees that the assembly is a simple acyclic join. Recent TD-based approaches have shown effectiveness for subgraph counting and labeled subgraph matching [2, 39, 40, 75]. However, unlabeled subgraph enumeration presents significantly greater challenges than both subgraph counting and labeled matching. The unlabeled setting generates exponentially more matches due to the absence of label constraints, while enumeration tasks require complete match materialization without the benefit of aggregation pushdown techniques available in counting. Consequently, substantial improvements to the tree decomposition framework are essential for effective unlabeled subgraph enumeration.

We identify three critical limitations in existing TD approaches.

① *suboptimal decomposition selection*: Existing approaches use brute-force enumeration or adopt a new branch-and-bound algorithm [25] to select the decomposition with the minimum fractional hypertree width. However, they only consider a single decomposition with the minimum width, while decompositions of identical width can exhibit dramatically different computational efficiency. ② *incomplete symmetry-breaking integration*: Integration of symmetry-breaking with tree decomposition has been explored only in subgraph counting [40], which only partially integrates symmetry-breaking with tree decomposition, focusing on rules compatible with aggregation pushdown. ③ *expensive materialization overhead*: After computing bag relations, TD approaches must join them to produce final results, which we term the global join. While subgraph counting approaches [40, 75] integrate the global join within bag processing via aggregation pushdown, subgraph matching approaches [2, 39] employ separate processing phases that incur high materialization costs to build complex data structures for all bags. Although existing work [39] proposed techniques to reduce memory usage for storing bag relations, it does not consider reducing the computational overhead for constructing such data structures. Furthermore, for bag attribute ordering, existing approaches rely on heuristics [2, 40, 75] or consider only intersection costs [39], ignoring that materialization overhead also depends on attribute ordering.

Main Contributions: To address these limitations, we present MDSE (Minimal Decomposition based Subgraph Enumeration) with four key contributions. First, we introduce minimal fractional hypertree decompositions (MinFHDs), where no bag can be replaced by smaller bags while maintaining correctness, and develop an efficient enumeration algorithm that explores all high-quality decompositions with minimum width rather than settling for a single decomposition. This approach ensures optimal computational efficiency while avoiding the scalability issues of brute-force methods. Second, we propose a systematic approach that integrates symmetry-breaking with the tree decomposition framework for subgraph enumeration. For a given set of symmetry-breaking rules, we show that we can utilize all of them rather than just a subset in subgraph matching. Since different symmetry rule sets have varying effectiveness, we propose an effective approach to select the optimal rule set that maximizes filtering opportunities. Third, we propose MixJoin, a novel algorithm that integrates the global join seamlessly within bag computation for subgraph matching. The algorithm significantly reduces bag materialization time cost, which is ignored in previous works. We develop a comprehensive

cost model that accounts for both intersection and materialization costs, enabling better attribute ordering decisions. Fourth, we conduct extensive experiments on 101 pattern graphs and 8 large real-world datasets to compare MDSE with state-of-the-art subgraph enumeration algorithms. MDSE consistently outperforms existing approaches, achieving up to 1-2 orders of magnitude speedup.

Organizations: We give preliminaries and problem formulation in Section 2. We introduce the tree decomposition framework in Section 3. In Section 4, we propose minimal fractional hypertree decompositions and present our efficient enumeration algorithm. We develop the systematic symmetry-breaking integration framework in Section 5. We propose the MixJoin algorithm and our cost model in Section 6. Section 7 discusses related works. We conduct comprehensive experimental studies and report the results in Section 8, and conclude this paper in Section 9.

2 PRELIMINARIES

Following the notation in [40], we model a simple undirected graph as $G = (V, E)$, where V and E are the sets of nodes and edges in G , respectively. A graph $G' = (V', E')$ is a subgraph of G if $V' \subseteq V$ and $E' \subseteq E$, and is an induced subgraph of G if E' contains all the edges in G such that both endpoints belong to V' . We call a graph a k -node graph (or k -graph) if it has k nodes. The number of nodes and the number of edges are denoted as $n = |V|$ and $m = |E|$.

Subgraph Isomorphism: Let $G = (V, E)$ be a data graph and $p = (V_p, E_p)$ be a pattern graph. A function $f : V_p \mapsto V$ is called a homomorphism of p if for each edge $(u, u') \in E_p$, we have $(f(u), f(u')) \in E$, and a homomorphism f of p is called a subgraph isomorphism of p if f is injective. That is, for any $u, u' \in V_p, u \neq u', f(u) \neq f(u')$. A subgraph $G_f = (V_f, E_f)$ in a data graph G is an *iso-match* of a pattern graph p if f is a subgraph isomorphism.

Automorphisms and Orbits: For a given graph $G = (V, E)$, an automorphism is a bijective function $\gamma : V \mapsto V$ such that $(v, v') \in E$ iff $(\gamma(v), \gamma(v')) \in E$. Here, an automorphism γ maps G to itself in a structure-preserving manner. The set of automorphisms, denoted as $\text{Aut}(G)$, forms a group called the automorphism group of G . With $\text{Aut}(G)$, two nodes $v, v' \in V$ are in an equivalence relation iff there exists an automorphism γ such that $\gamma(v) = v'$ (or $v \rightarrow v'$) for $\gamma \in \text{Aut}(G)$. Such equivalence relations partition nodes into equivalence classes, where an equivalence class is called an automorphism orbit, and is denoted as ϑ .

Problem Statement: In this work, we study unlabeled subgraph enumeration. Given an unlabeled pattern graph p and an unlabeled data graph G , subgraph enumeration returns all distinct iso-matches of p in G . It is different from listing subgraph isomorphisms as there are multiple subgraph isomorphisms for the same iso-match due to automorphisms of p .

Example 2.1: Fig. 1 illustrates the subgraph enumeration problem. Given the pattern graph p and the data graph G , we identify two distinct iso-matches of p in G , highlighted by green and purple dashed lines respectively. For the green iso-match, there are two different subgraph isomorphisms: $f_1 = \{u_0 \mapsto v_0, u_1 \mapsto v_1, u_2 \mapsto v_2, u_3 \mapsto v_3, u_4 \mapsto v_4, u_5 \mapsto v_5, u_6 \mapsto v_6\}$ and $f_2 = \{u_0 \mapsto v_1, u_1 \mapsto v_0, u_2 \mapsto v_2, u_3 \mapsto v_3, u_4 \mapsto v_4, u_5 \mapsto v_5, u_6 \mapsto v_6\}$. These arise due to pattern symmetry: p has an automorphism that swaps u_0 with u_1 . Thus $\{u_0, u_1\}$ forms an automorphism orbit.

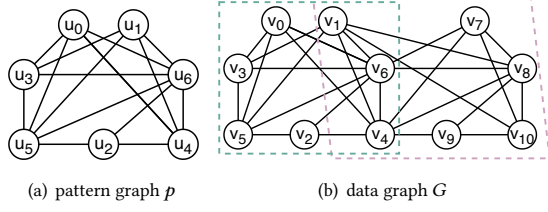


Figure 1: An example of subgraph enumeration

Tree decomposition is a powerful tool to decompose and process a pattern graph [17]. It is used for subgraph matching [2, 39], subgraph counting [40, 75] and theoretical approaches [16, 44]. The tree decomposition framework for subgraph matching is summarized in Algorithm 1.

Tree decomposition (line 1). A hypergraph is defined as $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of nodes and $\mathcal{E}$ is a set of hyperedges, where a hyperedge $e \in \mathcal{E}$ is a subset of $\mathcal{V}$. A pattern graph $p = (V_p, E_p)$ is a special hypergraph where every hyperedge only has two nodes. Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, a tree decomposition is a tree $T = (V_T, E_T)$ where each tree node $\tau_i \in V_T$ maintains a bag $\mathcal{V}(\tau_i) \subseteq \mathcal{V}$ with which an induced subgraph of $\mathcal{H}$ can be constructed, denoted as $\mathcal{H}(\tau_i)$. The tree satisfies the following conditions: (1) Every node and edge in $\mathcal{H}$ is covered by at least one bag; (2) For any vertex $v \in \mathcal{V}$, bags containing v form a connected subtree (running intersection property). We use $V_S(\tau_i)$ to denote the nodes in bag τ_i that also appear in neighbors of τ_i .

A fractional hypertree decomposition (FHD) is (T, ρ) where ρ assigns each bag τ_i a fractional edge cover value $\rho(\tau_i)$ [20]. The width of (T, ρ) is $\max_{\tau_i \in T} \rho(\tau_i)$, and the fractional hypertree width $\text{fhtw}(\mathcal{H})$ is the minimum width among all FHDs. For a pattern graph p , subgraph matching complexity is bounded by $O(|E|^{\text{fhtw}(p)} + \text{OUT})$ where OUT is the output size [2, 17]. Existing works select decompositions with minimum $\text{fhtw}(p)$.

Example 2.2: Fig. 2(a) shows an FHD T_1 of the pattern graph p in Fig. 1(a). The decomposition T_1 consists of three bags: τ_0, τ_1, τ_2 , each representing an induced subgraph of p . The decomposition satisfies all tree decomposition properties: every node and edge of the pattern graph is contained in at least one bag, and for each node, the bags containing that node form a connected subtree. For bag τ_0 , the shared nodes u_4, u_5 and u_6 appear in multiple bags, yielding $V_S(\tau_0) = \{u_4, u_5, u_6\}$. The induced subgraph $p(\tau_0)$ over nodes $\{u_5, u_6, u_2, u_4\}$ contains 5 edges, as illustrated in Fig. 2(a). The fractional hypertree width of T_1 is 2.5, which is the minimum among all FHDs of p , and $\text{fhtw}(p) = 2.5$.

Symmetry-breaking (line 2). Symmetry-breaking reduces redundant iso-matches when pattern graphs contain automorphisms [19, 50, 56, 57, 60, 68]. For an automorphism orbit $\vartheta = \{u_1, u_2, \dots, u_k\}$, a symmetry-breaking rule $\theta = \{u_1 < u_2, \dots, u_1 < u_k\}$ imposes ordering constraints. An iso-match f is valid if $f(u_1) < f(u_i)$ for all $u_1 < u_i$ in θ . In Example 2.1, we can apply a symmetry-breaking rule $u_0 < u_1$, which filters the redundant mapping f_2 .

Attribute order and attribute tree (line 4). A join attribute order (JAO) defines a total order of pattern graph nodes. WCOJ processes subgraph matching by incrementally extending partial matches following JAO. Emptyheaded, DISC and SCOPE select

Algorithm 1: The tree decomposition framework

Input: pattern graph p , data graph G
Output: subgraph matches of p in G

- 1 $T \leftarrow$ select a tree decomposition with minimum width;
- 2 Select symmetry-breaking rules;
- 3 **foreach** bag $\tau_i \in T$ **do**
- 4 select a JAO of $V(\tau_i)$;
- 5 Compute and materialize $R(\tau_i)$;
- 6 **output** $\bowtie_{\tau_i \in T} R(\tau_i)$;

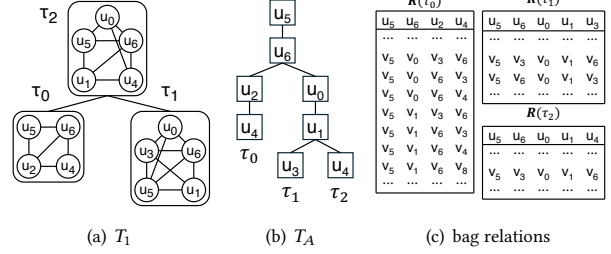


Figure 2: FHD, attribute tree and bag relations

JAOs by heuristics. ASDMatch proposes an attribute tree T_A where the JAO of each bag τ_i is represented as a root-to-leaf path, enabling shared computation among overlapping attributes. It selects an attribute tree by minimizing the cost for computing set intersections.

Join processing (lines 5-6). Tree decomposition approaches store data graph G in edge table R_g where each undirected edge (v, v') is stored as two tuples (v, v') , (v', v) and interpret pattern p as a self-join on R_g . For each bag τ_i , it computes matches of the induced subgraph $p(\tau_i)$ and materializes them in relation $R(\tau_i)$. After materializing all bag relations, it computes final matches by joining all $R(\tau_i)$, which is an acyclic join. DISC [75] and SCOPE [40] use binary joins with aggregation pushdown, integrating this step into bag processing. Emptyheaded [2] uses Yannakakis' algorithm [73] with complexity $O(IN + OUT)$, where $IN = O(|E|^{\text{fhtw}(p)})$ and OUT is the output size. ASDMatch [39] uses WCOJ for practical efficiency.

Example 2.3: Fig. 2(b) shows an attribute tree T_A constructed from the FHD T_1 in Fig. 2(a). Each root-to-leaf path in T_A represents a JAO for one of the bags. ASDMatch processes the three bag relations following their respective JAOs while sharing computations for overlapping attribute sequences. For instance, the leftmost path u_5, u_6, u_2, u_4 serves as the JAO for bag τ_0 . $R(\tau_0)$ is computed by executing the join:

$$R_g(u_2, u_4) \bowtie R_g(u_2, u_5) \bowtie R_g(u_2, u_6) \bowtie R_g(u_4, u_6) \bowtie R_g(u_5, u_6)$$

In the attribute tree T_A , nodes u_5, u_6 are shared among all three bags, while u_0, u_1 are shared between τ_0 and τ_1 . By reusing intermediate results, the matches of $p(\{u_5, u_6\})$ are computed once and shared across all three bags, and the matches of $p(\{u_5, u_6, u_0, u_1\})$ are shared between τ_0 and τ_1 . Using the data graph G from Fig. 1(b), some tuples of bag relations are shown in Fig. 2(c). For example, the tuple $[v_5, v_0, v_3, v_6]$ in $R(\tau_0)$ represents an iso-match of the induced subgraph $p(\tau_0)$ in G . Finally, we compute the join query $R(\tau_0) \bowtie R(\tau_1) \bowtie R(\tau_2)$ to obtain all subgraph matches of p in G .

Limitations of existing approaches. While tree decomposi-

Algorithm 2: MDSE (p, G)

Input: pattern graph p , data graph G
Output: subgraph matches of p in G

```

1  $T_A^* \leftarrow \emptyset, \Theta^* \leftarrow \emptyset, \text{minCost} \leftarrow \infty;$ 
2  $T \leftarrow \text{MinFHDs of } p \text{ with minimum width};$ 
3 foreach  $T \in T$  do
4    $\Theta \leftarrow \text{SelectSymmetries}(p, T);$ 
5    $T_A \leftarrow \text{select an attribute tree for } T, \Theta \text{ using our cost model};$ 
6   if  $\text{cost}(T_A) < \text{minCost}$  then
7      $T_A^* \leftarrow T_A, \Theta^* \leftarrow \Theta, \text{minCost} \leftarrow \text{cost}(T_A);$ 
8  $\text{MixJoin}(p, T_A^*, \Theta^*, G);$ 

```

Table 1: Summary of tree-decomposition-based approaches

Algorithm	decomposition selection	symmetry breaking	global join	attribute order
Emptyheaded [2]	brute-force	$\times$	extra step	heuristic
DISC [75]	brute-force	$\times$	within bag	heuristic
SCOPE [40]	brute-force	some	within bag	heuristic
ASDMatch [39]	branch-&-bound	$\times$	extra step	intersection cost
MDSE	minimal elimination order	all	within bag	intersection and materialization cost

tion is effective for labeled subgraph matching and subgraph counting, existing approaches have three key limitations for unlabeled subgraph enumeration. **Limitation 1: suboptimal decomposition selection.** Existing approaches compute only one decomposition with minimum fractional hypertree width [25]. However, decompositions with identical width can have vastly different computational costs. For instance, consider the pattern graph p in Fig. 1(a). On top of the FHD T_1 in Example 2.2, there are two other tree decompositions T_2 in Fig. 3(a) and T_3 in Fig. 3(b) with the same width, and their processing time varies between 119 seconds and 1,079 seconds. Simply selecting one decomposition misses many potentially better decompositions. **Limitation 2: incomplete symmetry-breaking.** Most decomposition-based algorithms ignore symmetry-breaking entirely, leading to redundant enumeration. SCOPE [40] applies symmetry-breaking but only uses rules compatible with its aggregation pushdown technique. For the pattern graph p in Fig. 1(a), nodes u_0 and u_1 are symmetric. However, for T_2 in Fig. 3(a) and T_3 in Fig. 3(b), SCOPE cannot apply the symmetry-breaking rule $u_0 < u_1$. **Limitation 3: expensive materialization cost.** Existing matching algorithms process bag relations and global joins as separate phases (lines 5-6 in Algorithm 1). Each bag relation $R(\tau_i)$ must be fully materialized with indexing structures before the global join. As shown in Example 2.3, this requires storing all tuples. The computation cost for materializing the tuples into indexing structures is overlooked in previous works.

3 AN OVERVIEW OF MDSE

To address the limitations identified in Section 2, we present MDSE (Minimal Decomposition-based Subgraph Enumeration). Algorithm 2 shows our framework, and Table 1 summarizes our improvements. Below we describe the technical challenges in overcoming each limitation and our solutions.

To overcome Limitation 1, we need to explore multiple tree decompositions. However, to our best knowledge, there is no efficient algorithm to enumerate multiple decompositions. Existing brute-force methods [2, 40, 75] enumerate all possible node subsets

as potential bags and all combinations of bags to enumerate decompositions, which is computationally prohibitive. We introduce *minimal* fractional hypertree decompositions (MinFHDs), where no bag can be replaced by smaller bags while preserving correctness. We show that MinFHDs achieve faster processing time and can be characterized through elimination orders, which enables efficient enumeration. In Algorithm 2, line 2 enumerates all MinFHDs, and lines 3-7 evaluate each candidate to select the best one.

To overcome Limitation 2, we need to fully apply the rules in a selected symmetry-breaking rule set. Symmetry-breaking rules come from symmetric (i.e. structurally equivalent by automorphism) nodes in p . However, symmetric nodes of p may not remain symmetric in $p(\tau)$ for a bag τ , making some rules inapplicable in SCOPE [40]. In our enumeration setting, we develop a predicate pushdown mechanism that can use any complete symmetry-breaking rule set, regardless of whether its symmetric nodes remain symmetric in bags. Besides, different symmetry-breaking rule sets lead to different predicate distributions, and evaluating which rule set minimizes computation requires considering their filtering effects. Our scoring mechanism (line 4 in Algorithm 2) evaluates each rule set based on the computational cost reduction at each bag, selecting the optimal one.

To overcome Limitation 3, we need to reduce the computation cost for building indexing structures for bag relations. Existing approaches [2, 39] store each bag relation $R(\tau_i)$ in a sorted trie data structure, whose irregular branching causes memory fragmentation and poor cache locality, making trie construction expensive. However, it is challenging to reduce the materialization while preserving the correctness and intersection efficiency of the algorithm. We propose the MixJoin algorithm with three techniques to reduce materialization overhead. First, we use *prefix attributes* that are processed like standard WCOJ, while only materializing the remaining attributes. This reduces trie levels without increasing intersection costs. Second, we identify a *containment condition* to completely avoid the materialization of the last bag to process. Third, we completely avoid building one-level tries. Our cost model (line 5 in Algorithm 2) evaluates both intersection and materialization cost to select the optimal attribute tree, and line 8 executes MixJoin with the selected configuration.

Remark. Although this paper focuses on unlabeled subgraph enumeration, our techniques are orthogonal to label-based filtering and can be integrated with labeled matching methods. (1) MinFHDs also apply in labeled settings because the decomposition structure is independent of labels, and compact bags still reduce intermediate results. (2) Labels may reduce but not eliminate pattern automorphisms (e.g., when multiple pattern nodes share the same label), so our predicate pushdown can still exploit residual symmetries. (3) MixJoin and our cost model also improve join processing in labeled matching, since materialization overhead remains.

4 MINIMAL DECOMPOSITIONS

To explore multiple high-quality decompositions efficiently, we introduce minimal fractional hypertree decompositions (MinFHDs).

Definition 4.1: A minimal fractional hypertree decomposition (MinFHD) of a hypergraph $\mathcal{H}$ is (T, ρ) where T is the tree decomposition and ρ is the fractional edge cover function, and there

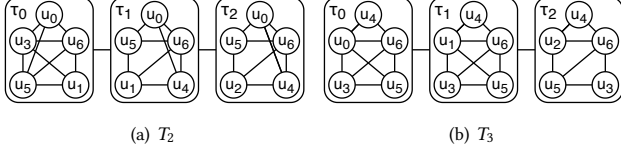


Figure 3: Different FHDs of the pattern graph p in Fig. 1(a)

does not exist another FHD (T', ρ') , such that there is a function $f : V_T \mapsto V_{T'}$, where $\forall \tau_i \in T, f(\tau_i) \subseteq \tau_i$, and $\exists \tau_j \in V_T, f(\tau_j) \subsetneq \tau_j$.

Example 4.1: The FHD T_1 in Fig. 2(a) is a MINFHD. Fig. 3(a) and Fig. 3(b) show two non-minimal FHDs with identical fractional hypertree width. In T_2 , bag τ_0 contains an additional node u_0 compared to T_1 , violating minimality. Similarly, T_3 is non-minimal since node u_3 can be removed from bag τ_2 in Fig. 3(b).

MINFHDs achieve computational advantages by ensuring each bag is fully utilized and cannot be replaced by smaller subsets. This compactness leads to smaller sub-queries and reduced intermediate results. Empirically, on the *yt* dataset, the three FHDs exhibit substantially different execution times: T_1 completes in 119s, T_2 requires 140s, and T_3 takes 1,079s, with minimal FHD T_1 achieving the best performance. We focus on finding an FHD that minimizes computational cost. This objective involves addressing two key challenges: finding FHDs with minimum width and ensuring minimality where bags cannot be further compacted. The following lemma establishes that an FHD satisfying both conditions exists—one that is both minimal and achieves minimum width.

Lemma 4.1: *Given a graph $p = (V_p, E_p)$, there exists a MINFHD (T, ρ) whose fractional hypertree width is $\text{fhtw}(p)$.*

The proofs in this paper are provided in the technical report [1].

Finding FHDs with minimum width has been addressed in [25]. We focus on handling the minimality condition. Direct enumeration of all MINFHDs is computationally expensive since it requires exploring the exponential space of possible decompositions while ensuring minimality conditions. To address this challenge efficiently, we establish a connection between MINFHDs and *minimal triangulations* [26] in graph theory, and leverage the *elimination order* technique to systematically enumerate all MINFHDs.

Triangulation and clique tree. A *triangulation* p^+ of p is a graph obtained by adding edges to p , i.e., $p^+ = (V_p, E_p \cup F)$ where F is a set of fill-in edges such that p^+ is chordal. A graph is chordal if every induced cycle in the graph is a triangle. A triangulation p^+ is called *minimal* if no proper subset of its fill-in edges produces a triangulation of p .

A *clique tree* of a triangulated graph p^+ is a tree where each node represents a maximal clique of p^+ , and the tree satisfies the running intersection property: for any vertex $v \in V_p$, the set of clique tree nodes containing v forms a connected subtree. The chordal graph p^+ has at most $|V_p|$ maximal cliques. A standard way to construct a tree decomposition of p is to triangulate p as p^+ , and then build a clique tree from the maximal cliques of p^+ . Because chordal graphs admit a clique tree whose nodes are the maximal cliques and satisfies the running intersection property, taking each maximal clique of p^+ as a bag and the edges of the clique tree yields a valid tree decomposition for p .

Example 4.2: Fig. 5(a) shows a minimal triangulation p_1^+ with fill-in edges $F = \{(u_0, u_1), (u_5, u_4)\}$ (dashed red). The resulting graph is chordal since no induced subgraph forms a cycle of length ≥ 4 . Fig. 5(b) displays the clique tree where each tree node represents a maximal clique of p_1^+ , corresponding to FHD T_1 in Fig. 2(a). In contrast, triangulation p_2^+ in Fig. 5(c) is non-minimal, corresponding to FHD T_2 in Fig. 3(a).

We show that the MINFHDs we proposed can be found using minimal triangulations.

Lemma 4.2: *Given a pattern graph $p = (V_p, E_p)$ and an FHD (T, ρ) of p , let $p_T^+ = (V_p, E_p \cup \bigcup_{\tau_i \in T, v \in \tau_i, v' \in \tau_i, v \neq v'} (v, v'))$. The FHD (T, ρ) is an MINFHD if and only if p_T^+ is a minimal triangulation of p .*

Having established the equivalence between MINFHDs and minimal triangulations, we now need a practical method to generate all minimal triangulations.

Elimination order. Given a pattern graph $p = (V_p, E_p)$, an *elimination order* π is a linear order of V_p . For simplicity, we use π_i to represent a node $v \in V_p$ as the i -th vertex in the linear order ($\pi(v) = i$). With a given linear order, a sequence of $|V_p| + 1$ graphs, $(p_\pi^0, p_\pi^1, \dots, p_\pi^{|\pi|})$, can be constructed for $p_\pi^+ = (V_p, E_\pi^+)$. Here, $p_\pi^0 = p$. $p_\pi^i = \text{elim}(p_\pi^{i-1}, \pi_i)$ where $\text{elim}(p, v)$ is an elimination operation that removes a vertex v from the input graph p , adds fill-in edges $D(p, v) = \{(v', v'') \mid v' \in N_p(v), v'' \in N_p(v), (v', v'') \notin E_p\}$ into p , and results in a new graph $p' = (V_p \setminus \{v\}, E_p \cup D(p, v))$. The last $p_\pi^{|\pi|}$ in the sequence is an empty graph.

Given an elimination order π , a triangulation p_π^+ can be produced by the elimination process as $p_\pi^+ = (V_p, E_p \cup \bigcup_{i=0}^{|\pi|-1} D(p_\pi^i, \pi_i))$ [26]. An elimination order π is minimal if there does not exist another elimination order π' such that $p_{\pi'}^+ \subsetneq p_\pi^+$.

Example 4.3: Fig. 4 demonstrates an elimination order $u_2, u_3, u_0, u_1, u_4, u_5, u_6$ for the pattern graph p in Figure 1(a). First, we perform the elimination operation $\text{elim}(p, u_2)$, which removes vertex u_2 from p and adds edges between all pairs of its neighbors that are not already connected. Since u_2 's neighbors are $\{u_4, u_5, u_6\}$ and vertices u_4 and u_5 lack an edge, we add the fill-in edge $D(p_0, u_2) = \{(u_4, u_5)\}$, yielding $p_1 = (V_p \setminus \{u_2\}, E_p \cup \{(u_4, u_5)\})$. Next, when eliminating u_3 , its neighbors in the remaining graph are $\{u_0, u_1, u_4, u_5, u_6\}$. Since u_0 and u_1 are not connected, we add the fill-in edge $D(p_1, u_3) = \{(u_0, u_1)\}$. The elimination process continues until we reach $p_7 = \emptyset$. This elimination order produces exactly the chordal graph p_1^+ shown in Figure 5(a), where $p_1^+ = (V_p, E_p \cup \{(u_4, u_5), (u_0, u_1)\})$. It is a minimal elimination order.

The following theorem guarantees that we can build all minimal triangulations as well as all MINFHDs using elimination orders.

Theorem 4.1: *Given a pattern graph $p = (V_p, E_p)$, p^+ is a minimal triangulation if and only if there exists a minimal elimination order of p such that $p^+ = p_\pi^+$. [49]*

Based on the established theoretical connections, we enumerate MINFHDs with minimum width $\text{fhtw}(p)$ using elimination orders. We first compute $\text{fhtw}(p)$ using [25], then enumerate all $|V_p|!$ permutations as elimination orders and use the clique trees of the corresponding triangulations as tree decompositions. We filter those that are not minimal or do not achieve minimum width. Due to

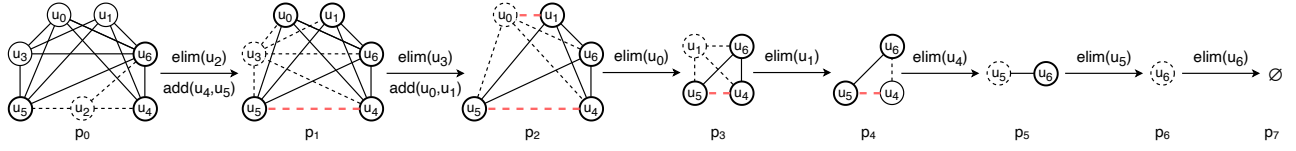


Figure 4: An elimination order $u_2, u_3, u_0, u_1, u_4, u_5, u_6$ for the pattern graph p in Fig. 1(a)

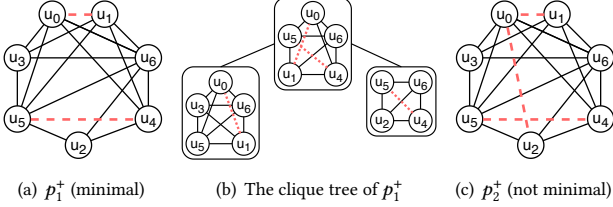


Figure 5: Triangulations of p in Fig. 1(a) and a clique tree

automorphisms, there may exist decompositions T, T' whose bags induce isomorphic subgraphs, i.e., there is a bijection $f : V_T \mapsto V_{T'}$ where $\forall \tau_i \in V_T, p(\tau_i)$ is isomorphic to $p(f(\tau_i))$, although they may not contain exactly the same pattern graph nodes. Since the computations are identical for such decompositions, we only keep one representative. Although exponential in $|V_p|$, this is efficient since pattern graphs are typically small.

5 APPLYING SYMMETRY-BREAKING

Symmetry-breaking eliminates redundant matches when pattern graphs contain automorphisms. For any given pattern graph, multiple symmetry-breaking rule sets exist and can be efficiently enumerated using the recursive approach from [40]. However, SCOPE can only use a subset of rules from the selected set that are compatible with its aggregation pushdown technique. In contrast, our enumeration-based approach can utilize a complete rule set. We treat symmetry-breaking rules as join predicates and partition them into local predicates that can be pushed down to individual bags and global predicates applied during final result assembly. Since different rule sets exhibit varying effectiveness due to different predicate distributions, we employ a scoring mechanism to select the optimal rule set.

5.1 Predicate Pushdown on Bags

In the unlabeled subgraph enumeration, there are two types of predicates, i.e., injectivity predicates and symmetry-breaking predicates. Injectivity predicates ensure that mappings of pattern graph nodes are distinct, as required by subgraph isomorphism. As established in Section 3, symmetry-breaking rules impose ordering constraints on automorphism orbits to eliminate redundant matches. For an automorphism orbit $\vartheta = \{u_1, u_2, \dots, u_k\}$, a symmetry-breaking rule $\theta = \{u_1 < u_2, \dots, u_1 < u_k\}$ ensures that iso-matches f satisfy $f(u_1) < f(u_i)$ for all $u_1 < u_i$ in θ . We define the complete predicate set $\mathcal{P}$ as the union of injectivity constraints and symmetry-breaking constraints:

$$\mathcal{P} = \{u_i \neq u_j \mid i \neq j, u_i, u_j \in V_p\} \cup \bigcup_{\theta \in \Theta} \{(u_i < u_j) \mid (u_i < u_j) \in \theta\} \quad (1)$$

where Θ is the set of symmetry-breaking rules. We denote the injectivity predicates as $\mathcal{P}^{inj} = \{u_i \neq u_j \mid i \neq j, u_i, u_j \in V_p\}$, which remain fixed regardless of Θ . Our focus is on efficiently handling the symmetry-breaking predicates $\mathcal{P}^{sym} = \bigcup_{\theta \in \Theta} \{(u_i < u_j) \mid (u_i < u_j) \in \theta\}$. We use $\phi_{i,j}$ to denote an individual predicate in $\mathcal{P}$ whose variables are u_i and u_j . In the following, we discuss how to use a symmetry-breaking rule set Θ .

With this unified predicate framework, subgraph enumeration can be expressed as a join over edge relations:

$$Q(p, \mathcal{P}) = \sigma_{\mathcal{P}} \left(\bowtie_{(u_i, u_j) \in E_p} R(u_i, u_j) \right) \quad (2)$$

where $\sigma_{\mathcal{P}}$ is the selection operator that filters matches based on predicates in $\mathcal{P}$.

In the tree decomposition framework, this query can be rewritten as:

$$Q(p, \mathcal{P}) = \sigma_{\mathcal{P}} \left(\bowtie_{\tau_i \in T} R(\tau_i) \right) \quad (3)$$

where $R(\tau_i) = \bowtie_{(u_k, u_l) \in E_{p(\tau_i)}} R(u_k, u_l)$ represents matches of subgraph τ_i . Applying predicates $\mathcal{P}$ as global filters after computing $\bowtie_{\tau_i \in T} R(\tau_i)$ is inefficient, requiring materialization before filtering and producing exponentially large intermediate results. Instead, we employ predicate pushdown [38, 70] to apply predicates early, significantly reducing intermediate relation sizes and computation cost. We can partition the predicate set $\mathcal{P}$ into local predicates $\mathcal{P}^{local}(\tau_i)$ for each bag and global predicates $\mathcal{P}^{global}$ for T . Local predicates are pushed down to individual bags. Global predicates are those where no bag contains both variables u_i and u_j .

$$\mathcal{P}^{local}(\tau_i) = \{\phi_{j,k} \in \mathcal{P} \mid u_j, u_k \in \tau_i\} \quad (4)$$

$$\mathcal{P}^{global} = \mathcal{P} \setminus \bigcup_{\tau_i \in T} \mathcal{P}^{local}(\tau_i) \quad (5)$$

With the predicate pushdown, a bag τ_i computes a relation $R_{\mathcal{P}}(\tau_i)$ that is smaller than $R(\tau_i)$.

$$R_{\mathcal{P}}(\tau_i) = \sigma_{\mathcal{P}^{local}(\tau_i)} \left(\bowtie_{(u_k, u_l) \in E_{p(\tau_i)}} R(u_k, u_l) \right) \quad (6)$$

This allows us to rewrite the overall query as:

$$Q(p, \mathcal{P}) = \sigma_{\mathcal{P}^{global}} \left(\bowtie_{\tau_i \in T} R_{\mathcal{P}}(\tau_i) \right) \quad (7)$$

Note that if a predicate $\phi_{i,j}$ can be pushed down to multiple bags, it is beneficial to apply the predicate to all such bags.

Example 5.1: For the pattern graph p in Fig. 6(a), where symmetric nodes are represented by the same color (blue and orange), we construct the FHD T shown in Fig. 6(b) with three bags: $\tau_0 = \{u_2, u_6, u_4, u_3, u_5\}$, $\tau_1 = \{u_0, u_6, u_2, u_3, u_5\}$, and $\tau_2 = \{u_1, u_6, u_4, u_3, u_5\}$. Consider the symmetry-breaking rule set $\Theta = \{u_3 < u_0, u_3 < u_1, u_3 < u_2, u_3 < u_4, u_2 < u_4, u_5 < u_6\}$. The figure illustrates how predicates in Θ can be pushed down to individual bags, with gray dashed lines connecting each predicate to the bags where it can be

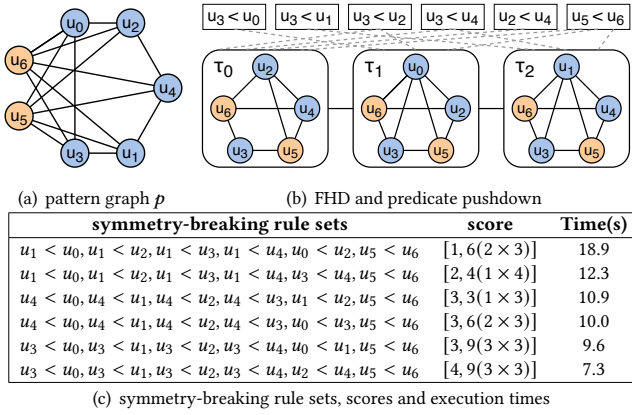


Figure 6: An example for pushing down and selecting *SymRs*

locally applied. For instance, the predicate $u_3 < u_0$ can be pushed down to τ_1 because both variables u_3 and u_0 are present in bag τ_1 .

Remark. Although each match of p is enumerated exactly once, our predicate pushdown approach does not guarantee that each match of $p(\tau_i)$ is enumerated exactly once. A bag relation $R(\tau_i)$ may lose some matches of $p(\tau_i)$ when symmetry-breaking predicates from $\mathcal{P}^{sym}$ are pushed down. For instance, in Fig. 6(b), u_3 and u_0 are not symmetric in τ_1 , but it still applies $u_3 < u_0$. However, since these matches cannot be extended to matches of $Q(p, \mathcal{P})$, it is safe to prune them early during bag computation. The counting-based approach SCOPE in [40] can only use *SymRs* that are compatible with its aggregation pushdown technique. A symmetry-breaking rule set Θ can be used if for each bag τ and each automorphism orbit ϑ in Θ , either $\tau \cap \vartheta = \emptyset$ or $\vartheta \subseteq \tau$ and ϑ is an automorphism orbit of $p(\tau)$. For example, for the pattern graph p in Fig. 1(a) and FHD T_1 in Fig. 2(a), a predicate $u_0 < u_1$ can be used since u_0 and u_1 are symmetric in $p(\tau_1)$ and $p(\tau_2)$, and are not present in τ_0 . In Fig. 6, none of the predicates $u_i < u_j$ in the table can be used by SCOPE due to these constraints. In contrast, since our method performs enumeration rather than counting, we do not require aggregation pushdown and can utilize any symmetry-breaking rule set from the candidates.

5.2 Selecting Symmetry-breaking Rules

Algorithm 3 presents our scoring mechanism for selecting the optimal symmetry-breaking rule set Θ^* from all candidate rule sets. It selects Θ^* that maximizes the number of local symmetry-breaking predicates for the most computationally expensive bags, thereby minimizing the computational overhead during query processing.

The algorithm operates in several key phases. First, we establish a canonical ordering of bags (line 1) by sorting them according to: (1) fractional hypertree width in descending order, as bags with larger width typically dominate computational cost; (2) for bags with identical width, the number of edges in the induced subgraph in ascending order, since fewer edges indicate more intermediate results where early filtering is more beneficial. Next, we group bags by graph isomorphisms to account for structural equivalence (line 3). For each candidate symmetry-breaking rule set Θ_i , we extract all binary comparison predicates $\mathcal{P}_i$ by flattening the partial orders

Algorithm 3: SelectSymmetries (p, T)

Input: pattern graph p , tree decomposition T with bags $\{\tau_1, \dots, \tau_k\}$
Output: selected symmetry-breaking rule set Θ

- 1 $\Theta_1, \dots, \Theta_n \leftarrow$ candidate symmetry-breaking rule sets of p ;
- 2 $\tau_{sorted} \leftarrow \text{SortBags}(\{\tau_1, \dots, \tau_k\})$;
- 3 $groups \leftarrow \text{GroupByGraphIsomorphism}(\tau_{sorted})$;
- 4 $bestScore \leftarrow (0, 0, \dots, 0), \Theta^* \leftarrow \emptyset$;
- 5 **foreach** $\Theta_i \in \{\Theta_1, \dots, \Theta_n\}$ **do**
- 6 $\mathcal{P}_i \leftarrow \bigcup_{\theta \in \Theta_i} \{(u_j < u_k) : (u_j < u_k) \in \theta\}$;
- 7 **foreach** $\tau_j \in \tau_{sorted}$ **do**
- 8 $localCount_j \leftarrow |\{(u_k < u_l) \in \mathcal{P}_i : u_k, u_l \in \tau_j\}|$;
- 9 $score_i \leftarrow \emptyset$;
- 10 **foreach** $group \in groups$ **do**
- 11 $groupCounts \leftarrow \{localCount_j : \tau_j \in group\}$;
- 12 $groupProduct \leftarrow \prod_{c \in groupCounts} c$;
- 13 $score_i \leftarrow score_i \cup \{groupProduct\}$;
- 14 **if** $score_i > bestScore$ **then**
- 15 $bestScore \leftarrow score_i$;
- 16 $\Theta^* \leftarrow \Theta_i$;
- 17 **return** Θ^* ;

within each symmetry-breaking rule $\theta \in \Theta_i$ (line 6), then compute the local predicate count for each bag τ_j , measuring how many predicates can be pushed down locally (lines 7-8). Within each group, local predicate counts are compared using a multiplicative scoring function to ensure predicates are distributed evenly across structurally equivalent bags, providing balanced computation reduction (lines 11-13). We compare score vectors in lexicographical order and prefer Θ_i that achieves higher scores for computationally challenging groups (lines 14-16).

Example 5.2: Continuing from Example 5.1, Fig. 6(c) demonstrates our algorithm's effectiveness. We sort the bags by computational complexity as τ_0, τ_1, τ_2 , where $p(\tau_1)$ and $p(\tau_2)$ induce isomorphic subgraphs and are grouped together during scoring. In Fig. 6(c), we evaluate six candidate symmetry-breaking rule sets with score vectors in the format $[localCount(\tau_0), groupProduct(\tau_1, \tau_2)]$, using the wiki-vote data graph. For instance, the notation $[1, 6(2 \times 3)]$ indicates that τ_0 can push down 1 predicate locally while τ_1 and τ_2 can push down 2 and 3 predicates respectively, yielding a group score of $2 \times 3 = 6$. The experimental results demonstrate a strong correlation between higher scores and superior performance. The optimal rule set with score $[4, 9(3 \times 3)]$ achieves the fastest execution time (7.3s), while the worst-scoring rule set $[1, 6(2 \times 3)]$ takes 18.9s, yielding a 2.6 $\times$ speedup. This validates our scoring mechanism: higher scores indicate more effective predicate pushdown, leading to reduced intermediate result sizes and faster query execution.

6 QUERY PROCESSING AND PLANNING

Previous methods [2, 39] process global join $\bowtie_{\tau_i \in T} R_{\mathcal{P}}(\tau_i)$ separately after computing bag relations $R_{\mathcal{P}}(\tau_i)$, requiring expensive materialization using complex data structures. Following ASD-Match [39], each bag relation $R_{\mathcal{P}}(\tau_i)$ is stored in a sorted trie to support efficient global joins. A sorted trie is a tree where each level corresponds to an attribute in τ_i , with each node storing multiple

Algorithm 4: MixJoin(p, T_A, Θ, G)

Input: pattern graph p , the attribute tree T_A for a TD T with bags $\{\tau_1, \dots, \tau_k\}$, symmetry-breaking rules Θ , data graph G

Output: Unique matches of p in G

```

1  $\mathcal{P} \leftarrow$  predicates by  $\Theta$  and  $T$ ; Pushdown  $\mathcal{P}$  on bags;
2 let  $\text{path}_S$  be the rightmost path of shared attributes of  $T_A$ ;
3 foreach  $i \in [1, k]$  do  $\text{prefix}_i \leftarrow \text{path}_i \cap \text{path}_S$ ;
4 foreach  $i \in [1, k]$ ,  $\text{prefix}_i = \emptyset$  do
5   | Compute and Materialize  $\mathbf{R}\mathcal{P}(\widehat{\tau}_i)$ 
6 let  $\pi_1, \dots, \pi_l$  be the order of  $\text{path}_S$ ;
7 TJoin( $\pi_1, V(G), \emptyset, l$ );
8 Procedure TJoin( $\pi_c, I_c, f, c$ )
9   while  $I_c \neq \emptyset$  do
10    |  $v \leftarrow \text{Inext}(I_c)$ ;  $I_c \leftarrow I_c \setminus \{v\}$ ;
11    |  $f \leftarrow f \cup \{\pi_c \mapsto v\}$ ;
12    | foreach  $i \in [1, k]$ ,  $\text{prefix}_i = [\pi_1, \dots, \pi_c]$  do
13      | if  $i \neq k$  then
14        | | Compute and Materialize  $\mathbf{R}\mathcal{P}(\widehat{\tau}_i)$ ;
15      | else if  $\bigcup_{\tau_i \in T} V_S(\tau_i) \subseteq \tau_k$  then
16        | | foreach  $f' \in \mathbf{R}\mathcal{P}(\widehat{\tau}_k)$  do
17          | | | foreach  $f'' \in f' \bowtie_{\tau_j \in T, j < k} \mathbf{R}\mathcal{P}(\widehat{\tau}_j)$  do
18            | | | | output  $f \cup f''$ ;
19        | | else
20          | | | Compute and Materialize  $\mathbf{R}\mathcal{P}(\widehat{\tau}_k)$ ;
21          | | | foreach  $f' \in \mathbf{R}\mathcal{P}(\widehat{\tau}_k)$  do
22            | | | | output  $f \cup f'$ ;
23      | if  $c < l$  then
24        | |  $I_{c+1} \leftarrow \text{Intersect}(\pi_{c+1}, f)$ ;
25        | | TJoin( $\pi_{c+1}, I_{c+1}, f, c+1$ );
26    |  $f \leftarrow f \setminus \{\pi_c \mapsto v\}$ ;

```

values representing possible extensions from partial tuples encoded by the root-to-node path. The irregular branching structure causes memory fragmentation and poor cache locality, making trie construction expensive. To reduce the materialization overhead, we propose the MixJoin algorithm and a comprehensive cost model.

6.1 Mixing Global and Local Joins

Algorithm 4 presents MixJoin to enumerate a pattern graph $p = (V_p, E_p)$ over a data graph $G = (V, E)$. It reduces materialization overhead while preserving intersection operations unchanged. We propose three techniques. First, we use some attributes at the front of the JAO of each bag as prefix attributes (prefix) and the remainder as materialized attributes. Prefix attributes are processed similarly as WCOJ for a single query, while materialized attributes follow the decomposition-based approach. Previous works [39, 40, 75] studied prefix usage but focused on reducing memory usage and their approach increases intersection cost. We focus on reducing materialization computation while keeping intersection operations identical to cases without prefix. Second, we identify a containment condition allowing processing the global join with the sub-query of the last bag, avoiding materializing its sorted trie. Third, we avoid materialization for bags with only one materialized attribute.

We use the notation $\widehat{\tau}_i = \tau_i \setminus \text{prefix}_i$ to denote materialized attributes of bag τ_i that require trie storage after processing prefix

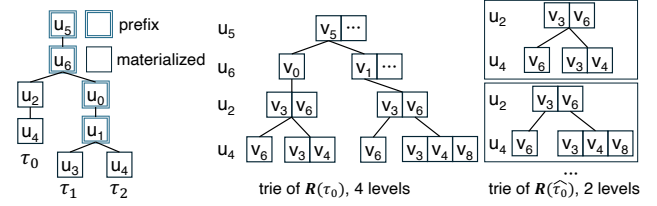


Figure 7: An example for prefix selection and MixJoin

attributes. Correspondingly, $\mathbf{R}\mathcal{P}(\widehat{\tau}_i)$ represents iso-matches for the materialized portion of bag τ_i .

Prefix Selection. Suppose the processing order of bags in the attribute tree is $\tau_1, \dots, \tau_k$. As discussed in [39], for a bag τ_i , the materialized attributes are processed for every match of $\bigcup_{j \leq i} \text{prefix}_j$. If $\bigcup_{j < i} \text{prefix}_j \subseteq \text{prefix}_i$, then the materialized attributes are processed for every match of prefix_i , ensuring that the intersections to compute τ_i remain identical to the case without prefix. To find such prefixes, our algorithm identifies the shared attributes in the attribute tree and uses its rightmost branch, denoted as path_S (line 2), to set the prefixes of each bag. Specifically, we compute $\text{prefix}_i = \text{path}_i \cap \text{path}_S$ for each bag τ_i (line 3). This construction guarantees that $\bigcup_{j < i} \text{prefix}_j \subseteq \text{prefix}_i$, preserving intersection behavior while maximizing the prefix usage. For bags where $\text{prefix}_i = \emptyset$ (lines 4-5), we immediately compute and materialize $\mathbf{R}\mathcal{P}(\widehat{\tau}_i)$ since no prefix optimization applies.

The core computation is handled by the recursive TJoin procedure, which traverses the shared attribute path path_S in order $\pi_1, \dots, \pi_l$ (line 6). For each attribute π_c and each candidate value v , the procedure extends the current partial mapping f and processes bags whose prefix matches the current path $[\pi_1, \dots, \pi_c]$ (lines 12-22). When reaching deeper levels ($c < l$), the procedure recursively continues with the next attribute π_{c+1} and its corresponding intersection results (lines 23-25). In lines 12-14, for a bag τ_i that is not the last one, we materialize $\mathbf{R}\mathcal{P}(\widehat{\tau}_i)$, thus reducing $|\text{prefix}_i|$ levels of trie materialization overhead.

Example 6.1: Fig. 7 illustrates prefix selection using attribute tree T_A from Fig. 2(b). The rightmost shared path $\text{path}_S = \{u_5, u_6, u_0, u_1\}$ determines prefix assignments: $\text{prefix}_0 = \{u_5, u_6\}$, $\text{prefix}_1 = \{u_5, u_6, u_0, u_1\}$, $\text{prefix}_2 = \{u_5, u_6, u_0, u_1\}$.

For each prefix match $\{u_5, u_6\}$ in bag τ_0 , we compute partial relation $\mathbf{R}(\widehat{\tau}_0)$ and materialize it as a two-level sorted trie. Fig. 7 compares trie structures for $\mathbf{R}(\tau_0)$ and $\mathbf{R}(\widehat{\tau}_0)$ using the data graph in Fig. 1(b) and the results in Fig. 2(c). The original trie requires 4 levels, while the optimized trie needs only 2 levels. This approach reduces materialization cost by limiting trie construction to non-prefix attributes.

Avoiding materialization for τ_k . The global join $\bowtie_{\tau_i \in T} \mathbf{R}\mathcal{P}(\tau_i)$ requires joining on $\bigcup_{\tau_i \in T} V_S(\tau_i)$, i.e., all attributes that appear in at least two bags, and traversing the remaining attributes. When the final bag τ_k contains all join attributes ($\bigcup_{\tau_i \in T} V_S(\tau_i) \subseteq \tau_k$), we can eliminate the materialization of $\mathbf{R}\mathcal{P}(\widehat{\tau}_k)$ entirely. When processing bag τ_k with a complete prefix match, we check the containment condition (line 15). If satisfied, we enumerate each tuple f' in $\mathbf{R}\mathcal{P}(\widehat{\tau}_k)$ on-the-fly (line 16) and immediately join it with previously computed bag relations, outputting final results directly

(lines 17-18). This streaming approach avoids materializing $\mathbf{R}_P(\widehat{\tau}_k)$ entirely, eliminating the expensive trie construction step. When the containment condition fails, we materialize $\mathbf{R}_P(\widehat{\tau}_k)$ (line 20) and perform the global join separately. With prefix, part of the global join $\bowtie_{\tau_i \in T} \mathbf{R}_P(\tau_i)$ is processed during bag computations, requiring joins only on attributes not in prefix. For unlabeled subgraph enumeration with typically small pattern graphs, our empirical analysis shows that 98 out of 101 queries satisfy the containment condition, making this optimization highly effective in practice.

Optimization for one-level trie. When $|\widehat{\tau}_i| = 1$, the materialized relation contains only one attribute, and we can reuse the intersection result I_c of that attribute without constructing an explicit trie. Since we store the data graph G using compressed sparse rows (CSR) format with sorted consecutive arrays for neighbor lists, the intersection operation naturally produces a sorted array I_c . When $|\widehat{\tau}_i| = 1$, the relation $\mathbf{R}_P(\widehat{\tau}_i)$ represents exactly those data graph nodes that can match the single remaining attribute given the current match of prefix _{i} , corresponding directly to I_c . This optimization eliminates 1-level trie construction.

Example 6.2: Continuing from Example 6.1, we demonstrate the one-level trie optimization for bag τ_1 and the containment condition optimization for bag τ_2 . Bag τ_1 has only one materialized attribute. In Algorithm 4, when we have matched $\{u_5, u_6, u_0, u_1\}$ and process τ_1 (line 14), we compute I_c for candidate matches of u_3 , which is reused as the one-level trie for τ_1 .

The global join $\mathbf{R}_P(\tau_0) \bowtie \mathbf{R}_P(\tau_1) \bowtie \mathbf{R}_P(\tau_2)$ requires joining on $V_S(\tau_0) \cup V_S(\tau_1) \cup V_S(\tau_2) = \{u_0, u_1, u_4, u_5, u_6\}$. Since these attributes are contained within final bag τ_2 , the containment condition $\bigcup_{\tau_i \in T} V_S(\tau_i) \subseteq \tau_2$ is satisfied, enabling streaming optimization. When processing bag τ_2 with prefix $\text{prefix}_2 = \{u_5, u_6, u_0, u_1\}$, partial mapping f covers these attributes while the remaining attribute $\widehat{\tau}_2 = \{u_4\}$ needs to be processed. Without materializing $\mathbf{R}_P(\widehat{\tau}_2)$, we enumerate matches f' for $\{u_4\}$ on-the-fly and immediately join with precomputed relations $\mathbf{R}_P(\widehat{\tau}_0)$ and $\mathbf{R}_P(\widehat{\tau}_1)$ to get matches of the pattern graph p extending the match of prefix₂. For each match of prefix₂, we generate new $\mathbf{R}_P(\widehat{\tau}_0)$ and $\mathbf{R}_P(\widehat{\tau}_1)$ and process the global join within τ_2 . In this way, we enumerate all matches incrementally.

6.2 The Cost Model

The MixJoin algorithm enables various attribute trees with different prefix selections and materialization strategies. To select the optimal attribute tree among all candidates, we develop a comprehensive cost model that considers both intersection operations and materialization overhead. We first introduce the intersection cost for an attribute in a JAO, which differs from [39, 47] since we also consider symmetry-breaking effects. Given a partial match p_{k-1} induced by $\pi[1..k-1]$ for some $k > 1$, to match p_k by extending $\pi[1..k-1]$ to $\pi[1..k]$, for each $(\pi_l, \pi_k) \in E_p$, where $l < k$, we intersect the neighbors of $f(\pi_l)$. The intersection operation can be represented as $\bigcap_{(\pi_l, \pi_k) \in E_p} N_G(f(\pi_l))$ where $N_G(f(\pi_l))$ are the neighbors of the data graph G . The neighbor list size is estimated as the average degree of G if there are no symmetry-breaking constraints. When symmetry-breaking constraints are applied, the neighbor list size is reduced due to the ordering restrictions imposed by predicates in $\mathcal{P}$. Specifically, if there exists an ordering constraint between π_l

and π_k , the symmetry-breaking predicates will filter out approximately half of the candidate neighbors. Therefore, we estimate the neighbor list size for each $l < k$, $(\pi_l, \pi_k) \in E_p$ as:

$$\mu_{\pi_k}^{\pi_l} = \begin{cases} |E(G)|/|V(G)| & \text{if } u_l < u_k \in \mathcal{P} \text{ or } u_k < u_l \in \mathcal{P}, \\ 2 \times |E(G)|/|V(G)| & \text{otherwise.} \end{cases} \quad (8)$$

Let $\mathbb{R}_{k-1}$ be a relation storing all partial iso-matches for $\pi[1 : k-1]$ that satisfy the predicates $\mathcal{P}$ involving only variables $\{\pi_1, \dots, \pi_{k-1}\}$, and $\mu(\mathbb{R}_{k-1})$ be the estimated cardinality of $\mathbb{R}_{k-1}$. Then the intersections for π_k are computed $\mu(\mathbb{R}_{k-1})$ times. The cost for computing each intersection is estimated as the total size of the sets. By multiplying the number of intersections, the cost is as follows.

$$\text{i-cost}(\pi_k) = \mu(\mathbb{R}_{k-1}) \sum_{l < k, (\pi_l, \pi_k) \in E_p} \mu_{\pi_k}^{\pi_l} \quad (9)$$

The intersection cost for an attribute tree $T_A = (V_A, E_A)$ is defined as the sum of individual costs:

$$\text{i-cost}(T_A) = \sum_{u \in V_A} \text{i-cost}(u) \quad (10)$$

The materialization cost. The materialization cost captures the overhead of building sorted tries to store intermediate relations. Different JAOs lead to different prefix selections prefix _{i} , resulting in varying materialization costs. A bag τ_i that is not the final bag (or the final bag when the containment condition fails) needs to materialize $\mathbf{R}_P(\widehat{\tau}_i)$. For each match of prefix _{i} , the algorithm materializes one instance of $\mathbf{R}_P(\widehat{\tau}_i)$. By summing over all matches of prefix _{i} , the total number of tuples across all materialized relations $\mathbf{R}_P(\widehat{\tau}_i)$ equals $\mu(\mathbf{R}_P(\tau_i))$. The materialization cost depends on the number of trie levels $|\widehat{\tau}_i|$. We have shown that when $|\widehat{\tau}_i| = 1$, the materialization can be saved by reusing intersection results. For multi-level tries ($|\widehat{\tau}_i| > 1$), explicit materialization becomes necessary.

Therefore, the materialization cost is defined as:

$$\text{m-cost}(\tau_i) = \begin{cases} 0 & \text{if } \tau_i = \tau_k, \bigcup_{\tau_j \in T} V_S(\tau_j) \subseteq \tau_k \text{ or } |\widehat{\tau}_i| = 1, \\ \mu(\mathbf{R}_P(\tau_i)) \times |\widehat{\tau}_i| & \text{otherwise.} \end{cases} \quad (11)$$

The total cost of processing an attribute tree combines both intersection and materialization costs:

$$\text{cost}(T_A) = \text{i-cost}(T_A) + \alpha \sum_{\tau_i \in T} \text{m-cost}(\tau_i) \quad (12)$$

We set the coefficient $\alpha = 20$ based on empirical evaluation and use the subgraph cardinality estimation algorithm in [58].

Example 6.3: We demonstrate cost calculation using the attribute tree T_A from Figure 7. Consider computing intersection cost for attribute u_3 during enumeration. For each partial match f of $p(\{u_5, u_6, u_0, u_1\})$, we compute candidate data graph nodes that can extend the match to include u_3 by intersecting neighbor sets $N_G(f(u_5))$, $N_G(f(u_6))$, $N_G(f(u_0))$, and $N_G(f(u_1))$ to obtain intersection result I_c in Algorithm 4. The number of such intersection operations equals $\mu(\mathbb{R}(\{u_5, u_6, u_0, u_1\}))$, which represents the estimated cardinality of $p(\{u_5, u_6, u_0, u_1\})$ after applying symmetry-breaking constraint $u_0 < u_1$. Each neighbor list size $|N_G(f(u_i))|$ for $i \in \{0, 1, 5, 6\}$ is estimated as $\frac{2|E_G|}{|V_G|}$ based on average degree in the data graph. Therefore, intersection cost for u_3 is computed as

$i\text{-cost}(u_3) = \mu(\mathbf{R}(u_5, u_6, u_0, u_1)) \times \frac{8|E_G|}{|V_G|}$. Intersection costs of other attributes follow similar computations according to Equation (9).

For materialization cost analysis, bag τ_1 materializes only the single attribute u_3 , yielding $m\text{-cost}(\tau_1) = 0$. Bag τ_2 serves as the final bag containing all shared attributes, enabling streaming optimization, so $m\text{-cost}(\tau_2) = 0$. However, bag τ_0 requires explicit materialization since $\widehat{\tau}_0 = \{u_2, u_4\}$ with $|\widehat{\tau}_0| = 2 > 1$, resulting in $m\text{-cost}(\tau_0) = 2 \times \mu(\mathbf{R}_{\mathcal{P}}(\tau_0))$.

Query Planning. With the cost model established, we select the optimal attribute tree from all valid candidates. The search space is large as we must consider final bag choice, $\mathcal{JAO}$ for each bag, and sharing opportunities among $\mathcal{JAO}$ s. We generate attribute trees from a restricted candidate set. First, we select a final bag from candidates that contain all shared attributes $\bigcup_{\tau_i \in T} V_S(\tau_i)$; if no such bag exists, we choose bags with largest $|V_S(\tau_i)|$. For each final bag choice, we construct the shared upper portion of the attribute tree (attribute nodes with ≥ 2 children) by combining permutations of each $V_S(\tau_i)$ and merging common prefixes. Remaining attributes within each bag are independent of other bags, with ordering following [7]. This process generates high-quality execution plans efficiently.

7 RELATED WORK

Subgraph Matching and Graph Mining Systems. Extensive research exists for subgraph enumeration on unlabeled graphs, spanning single-machine approaches [19, 35, 60] and distributed systems [36, 37, 50, 52, 72]. LIGHT [60] represents the state-of-the-art single-machine algorithm, employing lazy materialization to defer pattern node matching and reduce set intersection overhead. Among distributed approaches, some works decompose patterns into sub-structures before assembling their matches [36, 37, 50, 52]. HuGE [72] expresses queries as multi-way joins over clique and star units, supporting flexible pushing/pulling nodes to balance memory and computational efficiency. Graph mining systems [4, 9–13, 21, 27, 28, 41, 56, 57, 64] extend to applications based on subgraph enumeration, like subgraph counting and frequent subgraph mining. GraphSet [56] exemplifies this approach, leveraging set properties to eliminate control flow overhead. There is also work studying subgraph matching for labeled graphs. Most follow the classic filtering-ordering-enumerating framework [61, 77]. Research efforts have been devoted to pre-computing and filtering the search space [5, 6, 14, 23, 24, 33, 34, 62, 79], selecting good matching orders [6, 23, 24, 33, 34, 47, 54, 62], pruning invalid search branches [3, 23, 43], reusing previous matching results [29, 33, 34, 42, 43], and developing new enumeration algorithms [31, 39, 42, 78]. Recent surveys and experimental studies can be found in [30, 61, 77].

Tree decomposition and Query Processing. Tree decomposition has been widely used for constraint satisfaction problems [18], cyclic conjunctive queries [17] and their extensions [67]. The state-of-the-art approach to computing optimal FHD is BB4FHD [25]. For applying tree decompositions to query processing, CacheTrieJoin [32] uses tree decomposition to accelerate WCOJ on input queries. EmptyHeaded [2] follows a classic two-phase approach that computes bags first and then joins materialized views of bags. ADJ [74] studies using the two-phase approach in distributed systems. DISC [75] and SCOPE [40] study subgraph counting by converting the count of p to a linear combination of the counts of other subpatterns

Table 2: Data graphs

Category	Dataset	Name	$ V $	$ E $	d
Web	Wiki-vote [71]	<i>wv</i>	7,115	100,762	28.3
Collaboration	Mico [15]	<i>mc</i>	96,638	1,080,156	22.4
Social	Youtube [71]	<i>yt</i>	1,134,890	2,987,624	5.3
Web	eu2005 [53]	<i>eu</i>	862,664	16,138,468	37.4
Citation	US Patents [71]	<i>up</i>	3,774,768	16,518,947	8.8
Social	Livejournal [71]	<i>lj</i>	4,846,609	42,851,237	17.7
Social	Orkut [71]	<i>ok</i>	3,072,441	117,185,083	76.2
Social	Friendster [71]	<i>fs</i>	65,608,366	1,806,067,135	55.1

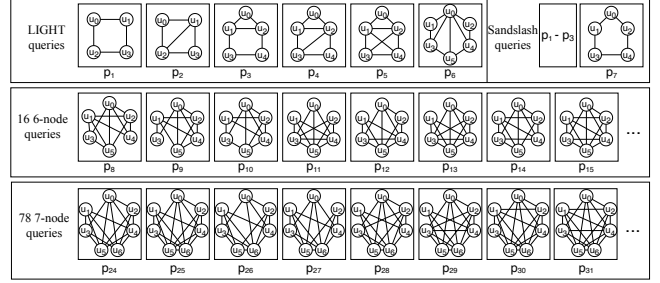


Figure 8: 101 pattern graphs

and processing each count by tree decomposition. ASDMatch [39] optimizes attribute ordering and proposes an adaptive approach to improve efficiency while bounding memory usage.

8 EXPERIMENTS

We conduct comprehensive experiments to evaluate the effectiveness of our MDSE algorithm.

8.1 Experimental Setup

Algorithms: We compare MDSE against four state-of-the-art baselines for subgraph enumeration. LIGHT [60] represents the state-of-the-art single-machine algorithm, employing a lazy materialization technique that defers pattern node matching until necessary to reduce computational overhead from set intersections. GraphSet [56] is a graph mining system. It incorporates sophisticated optimization techniques including a cost model for attribute ordering and symmetry-breaking selection inherited from GraphPi [57] and the elimination of redundant control flows via set properties. It does not use query decomposition. GraphZero [45] is a graph mining system that uses symmetry-breaking and generalizes orientation optimization to arbitrary patterns. Sandlash [11] is a graph mining system that automatically selects exploration order and data representations given only a pattern specification.

Datasets: We evaluate our approach on 8 large-scale real-world graphs commonly used in previous subgraph enumeration studies. Table 2 presents detailed statistics, including node counts, edge counts, and average degrees. Our dataset collection spans diverse graph types. The majority of datasets (*wv*, *yt*, *up*, *lj*, *ok*, *fs*) are sourced from [71], while *mc* and *eu* are obtained from [15] and [53] respectively. Following standard preprocessing practices, we treat all edges as undirected and remove isolated nodes and self-loops from the original dataset graphs.

Pattern graphs: Our evaluation encompasses 101 pattern graphs

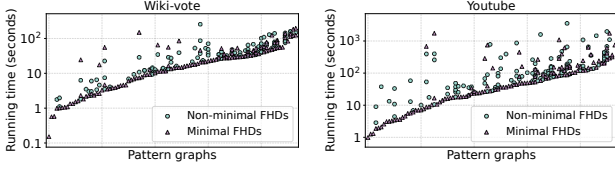


Figure 9: Comparing minimal and non-minimal FHDs

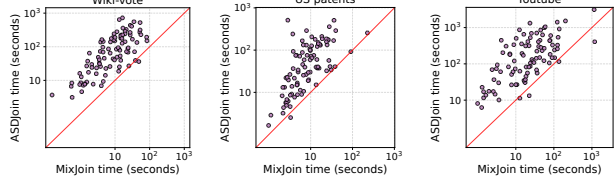


Figure 10: Effect of MixJoin

Table 3: Performance of MinFHD enumeration (time in ms)

# nodes	6	6	6	7	7	7	7	7
# edges	11	12	13	15	16	17	18	19
pattern count	9	5	2	40	21	10	5	2
Brute-force [40]	24.1	15.7	0.9	1184	661	90.6	15.4	1.0
MDSE	2.2	2.1	1.5	11.1	7.1	5.3	5.2	2.2

Table 4: Average Speedup of optimal SymRs vs. alternatives

Approaches	Wiki-vote		Youtube		Patents	
	k = 6	k = 7	k = 6	k = 7	k = 6	k = 7
no SymRs	6.38×	8.72×	4.84×	5.83×	4.02×	6.75×
partial SymRs	1.30×	1.96×	1.89×	3.43×	1.55×	2.66×
slowest full SymRs	1.49×	1.69×	3.00×	1.81×	1.27×	1.50×

ranging from 4 to 7 nodes, as illustrated in Fig. 8. Due to their specialized implementations, LIGHT and Sandslash support only limited pattern sets. LIGHT handles p_1 - p_6 , while Sandslash supports p_1 - p_3 and p_7 . We omit clique patterns from LIGHT since tree decompositions provide limited advantages for these structurally simple cases. LIGHT, Sandslash and our MDSE implement ad-hoc optimizations for these specific patterns. GraphSet and GraphZero support arbitrary pattern graphs, though existing evaluation [56] covered only 6 patterns for subgraph enumeration. To ensure a comprehensive comparison, we conduct extensive evaluation against GraphSet and GraphZero using 94 patterns without ad-hoc optimizations. These patterns include 16 six-node patterns with 11-13 edges and 78 seven-node patterns with 15-19 edges. We exclude patterns with very sparse edges, which generate prohibitively large results, and cliques or cliques missing a single edge, as these algorithms handle these cases with similar efficiency.

Environment: All experiments are conducted on a computing platform running CentOS 9.4, equipped with an Intel Xeon E7-8891 v3 processor featuring 80 cores and 256GB of memory. All algorithms are implemented in C++ and compiled using g++ 11.4.1 with aggressive optimization (-O3) to achieve optimal performance. For parallel execution, all algorithms use 64 threads.

8.2 MDSE Testing

MinFHD enumeration. We compare our elimination order-based MinFHD enumeration against the brute-force method with pruning optimizations from [40]. Our method achieves 194.8× average

speedup across all 94 patterns. Critically, while the brute-force approach reaches 9182.8ms maximum time, which exceeds the total execution time of many actual enumeration tasks, our approach achieves a maximum time of only 35.5ms, maintaining negligible decomposition overhead. Table 3 shows detailed MinFHD enumeration results grouped by pattern sizes. Our approach achieves 14.7× speedup for 6-node patterns (16 instances) with average times of 2.1ms vs. 18.6ms, and 231.8× speedup for 7-node patterns (78 instances) with average times of 8.7ms vs. 798.2ms. For the two cases where brute-force is faster (6-node 13-edge and 7-node 19-edge patterns), these patterns contain large non-decomposable clique components. The brute-force method includes optimizations that detect such cliques and directly solve the remaining smaller subproblem, bypassing much of the enumeration. In contrast, our approach still performs elimination order enumeration and triangulation computation for these patterns. However, these cases are acceptable, as the performance difference is small, and they are simple to compute.

Efficiency of MinFHDs. Fig. 9 demonstrates the performance characteristics of minimal versus non-minimal FHDs across 94 patterns on two real-world datasets. For each pattern, we evaluate all available tree decompositions with the minimum width and classify them as minimal or non-minimal. There are 37 patterns that only have one decomposition, and 57 patterns have multiple decompositions. Patterns are sorted by their fastest execution time. Across both datasets, minimal decompositions (triangular points) achieve faster execution times compared to non-minimal alternatives (round points). This trend is particularly evident in the lower regions of both plots, where the fastest decompositions are predominantly minimal. Quantitatively, MinFHDs achieve the fastest execution time in 86.2% of cases on *wv* and 84.0% on *yt*. Even when non-minimal decompositions are faster, the performance gap is modest. On average, the fastest MinFHD is 1.3× faster than the fastest non-minimal FHD on *wv*, and 2.6× faster on *yt*. More significantly, using the worst decomposition instead of the best MinFHD results in substantial slowdowns of 2.1× on *wv* and 7.3× on *yt*, highlighting the importance of careful decomposition selection. There are some triangle points for MinFHDs with exceptionally long execution time. We explain it as follows. For two FHDs T_1, T'_1 that both have the minimum width, T_1 is minimal, T'_1 is not minimal and can be built by enlarging some bags in T_1 , and T'_1 is slower than T_1 in general. However, T'_1 can be faster than another MinFHD T_2 . An FHD T'_2 made by enlarging some bags of T_2 could be slower than T_2 but is not included in the figure since its width becomes larger. In such cases, despite being minimal, T_2 appears as the slowest point for the pattern graph. Moreover, note that these cases are in the minority, and we can avoid bad MinFHDs by choosing faster MinFHDs using the cost model (Algorithm 2).

Effect of SymRs pushdown and selection. Table 4 compares our optimal symmetry selection against three alternatives: no symmetry-breaking, partial symmetry-breaking (as in [40]), and worst selection of full symmetry-breaking. Optimal selection achieves substantial speedups: 4.02×–8.72× vs. no symmetry-breaking, 1.30×–3.43× vs. partial symmetry, and 1.27×–3.00× vs. worst selection. This is because our predicate pushdown technique allows each bag to utilize more SymRs than prior work, and our scoring mechanism

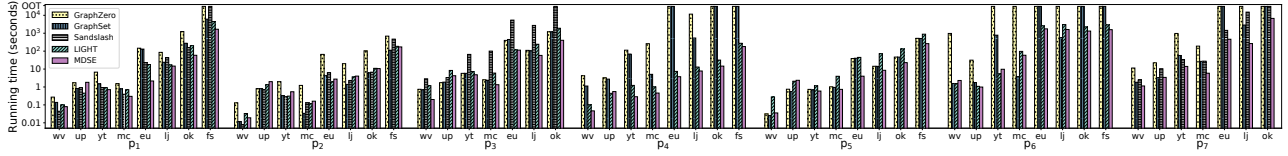


Figure 11: Evaluation on specific pattern graphs

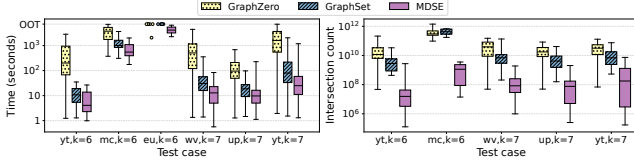


Figure 12: Evaluation on general pattern graphs

selects superior *SymRs* by maximizing local symmetry-breaking predicates for the most computationally expensive bags.

Effect of MixJoin. Fig. 10 compares our MixJoin algorithm (Sec. 6) against ASDJoin [39] across three datasets. The key difference is that MixJoin processes the global join within the final bag computation while ASDJoin requires an extra step. Each scatter plot shows execution times where the x-axis represents MixJoin time and the y-axis represents ASDJoin time. Points above the diagonal $y = x$ indicate cases where MixJoin is better. The results demonstrate overwhelming advantages for MixJoin, with the vast majority of points positioned above the diagonal across all datasets. MixJoin achieves remarkable average speedups of 11.6 $\times$ on *wv*, 9.2 $\times$ on *up*, and 12.8 $\times$ on *yt*. These results validate that MixJoin’s integration of global joins with bag computation significantly reduces materialization overhead while preserving set intersection efficiency.

Additional experiments on per-pattern symmetry score impact, the effect of materialization-aware cost modeling, and a detailed ablation study are provided in the technical report [1].

8.3 Performance Evaluation

Evaluation on Specific Patterns. Fig. 11 compares all five algorithms on specific patterns p_1 – p_7 across multiple datasets. The y-axis shows execution time with a 6-hour timeout. LIGHT supports p_1 – p_6 and Sandslash supports p_1 – p_3 and p_7 , while GraphZero, GraphSet, and MDSE support all patterns. Among the baselines, LIGHT is the most competitive. MDSE achieves competitive performance with LIGHT on p_2 and demonstrates clear advantages on other patterns, most notably 5.6 $\times$ average speedup for p_5 and 3.4 $\times$ for p_3 . Against Sandslash, MDSE achieves 11.6 $\times$ average speedup. On complex patterns p_1 , p_3 and p_7 , Sandslash times out on four large datasets while MDSE completes all these instances. GraphZero and GraphSet are relatively slow since they do not implement ad-hoc optimizations for these specific patterns, and MDSE outperforms them by more than two orders of magnitude. The speedups are less significant on *up*, which is sparser and has a more uniform degree distribution. As a result, large patterns (e.g., the full pattern) have fewer matches in *up*. Smaller patterns (e.g., pattern subgraphs within bags) still have many matches, leading to a high materialization cost as we need to store them in complex data structures. When the data graph is very sparse and subgraph matches are rare,

matching the full pattern directly (as the baselines do) is already efficient, and the advantage of first matching smaller bags and joining results becomes less pronounced.

Evaluation on General Patterns. Fig. 12 presents a comprehensive comparison of GraphZero, GraphSet, and MDSE across different datasets and pattern graphs. The left part shows the running time. The timeout is 2 hours. The right part shows the number of set intersections to enumerate all subgraph matches, a metric to evaluate the cost of the set intersection computation in previous works [30, 31, 39, 61, 62]. For a fair comparison, we collect this count for instances where all three algorithms can finish within the time limit. The results demonstrate consistently strong MDSE advantages. On *eu*, MDSE can run six 6-node patterns while GraphSet cannot run any 6-node pattern in the time limit, and GraphZero can only handle one pattern. In other cases, MDSE outperforms GraphSet by 2.0 $\times$ –7.7 $\times$ and outperforms GraphZero by 9.3 $\times$ –156 $\times$. MDSE also significantly reduces the number of set intersections by 1,605 $\times$ –4,139 $\times$ compared to GraphSet and 185 $\times$ –19,424 $\times$ compared to GraphZero. In our decomposition-based approach, we enumerate $p(\tau_i)$ rather than p . When joining $R(\tau_1) \bowtie \dots \bowtie R(\tau_k)$, we only need to join on attributes that are contained in multiple bags. Therefore, MDSE can use fewer set intersections to compute the results. It is the key reason that MDSE outperforms the baselines. The difference in running time is less significant than that of the intersection count. Besides, in some cases MDSE can be slower than GraphSet. This is because tree decomposition-based approaches inherently incur *materialization cost*. After computing subgraph matches for each bag, intermediate results must be stored as sorted tries before performing the global join (Section 6). In contrast, GraphSet and GraphZero directly enumerate matches without constructing any intermediate data structures, and thus have no materialization cost. When the materialization cost is large and unavoidable, this overhead can outweigh the benefits of decomposition, allowing GraphSet to outperform MDSE.

In our technical report [1], we also confirm MDSE’s strong parallelizability through thread scaling experiments.

9 CONCLUSION

We propose a novel decomposition-based approach MDSE for unlabeled subgraph enumeration. We introduce minimal fractional hypertree decompositions (MinFHDs) and develop an efficient elimination order algorithm to enumerate all MinFHDs with minimum width. We integrate symmetry-breaking with tree decomposition. Additionally, we design the MixJoin algorithm to reduce materialization overhead, and develop a comprehensive cost model that considers both intersection and materialization costs for optimal attribute ordering. Extensive experiments on eight data graphs demonstrate that MDSE consistently outperforms four state-of-the-art algorithms. On 7 selected patterns, MDSE achieves over two orders of magnitude speedup against GraphSet and GraphZero,

11.6× against Sandslash, and up to 5.6× against LIGHT. On 94 general 6/7-node patterns, MDSE outperforms GraphSet by 2.0×–7.7× and GraphZero by 9.3×–156×.

REFERENCES

- [1] Technical report of this paper. <https://github.com/magic62442/subgraph-enumeration>.
- [2] C. R. Aberger, S. Tu, K. Olukotun, and C. Ré. Emptyheaded: A relational engine for graph processing. In *SIGMOD*, pages 431–446. ACM, 2016.
- [3] J. Arai, Y. Fujiwara, and M. Onizuka. Gup: Fast subgraph matching by guard-based pruning. *Proceedings of the ACM on Management of Data*, 1(2):1–26, 2023.
- [4] A. Arpaci-Dusseau, Z. Zhou, and X. Chen. Accurate and fast approximate graph pattern mining at scale. *Proc. VLDB Endow.*, 18(2):93–107, 2024.
- [5] B. Bhattarai, H. Liu, and H. H. Huang. CECI: compact embedding cluster index for scalable subgraph matching. In *SIGMOD*, pages 1447–1462. ACM, 2019.
- [6] F. Bi, L. Chang, X. Lin, L. Qin, and W. Zhang. Efficient subgraph matching by postponing cartesian products. In *SIGMOD*, pages 1199–1214. ACM, 2016.
- [7] V. Bonnici, R. Giugno, A. Pulvirenti, D. E. Shasha, and A. Ferro. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinform.*, 14(S-7):S13, 2013.
- [8] K. Bringmann, N. Carmeli, and S. Mengel. Tight fine-grained bounds for direct access on join queries. *ACM Transactions on Database Systems*, 50(1):1–44, 2025.
- [9] H. Chen, M. Liu, Y. Zhao, X. Yan, D. Yan, and J. Cheng. G-miner: an efficient task-oriented graph mining system. In *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23–26, 2018*, pages 32:1–32:12. ACM, 2018.
- [10] X. Chen and Arvind. Efficient and scalable graph pattern mining on gpus. In M. K. Aguilera and H. Weatherspoon, editors, *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11–13, 2022*, pages 857–877. USENIX Association, 2022.
- [11] X. Chen, R. Dathathri, G. Gill, L. Hoang, and K. Pingali. Sandslash: a two-level framework for efficient graph pattern mining. In H. Zhou, J. Moreira, F. Mueller, and Y. Etsion, editors, *ICS '21: 2021 International Conference on Supercomputing, Virtual Event, USA, June 14–17, 2021*, pages 378–391. ACM, 2021.
- [12] X. Chen, R. Dathathri, G. Gill, and K. Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proc. VLDB Endow.*, 13(8):1190–1205, 2020.
- [13] X. Chen, T. Huang, S. Xu, T. Bourgeat, C. Chung, and Arvind. Flexminer: A pattern-aware accelerator for graph pattern mining. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14–18, 2021*, pages 581–594. IEEE.
- [14] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento. A (sub)graph isomorphism algorithm for matching large graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 26(10):1367–1372, 2004.
- [15] M. Elseidy, E. Abdelhamid, S. Skiadopoulos, and P. Kalnis. GRAMI: frequent subgraph and pattern mining in a single large graph. *Proc. VLDB Endow.*, 7(7):517–528, 2014.
- [16] J. Flum and M. Grohe. *Parameterized Complexity Theory*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2006.
- [17] G. Gottlob, G. Greco, N. Leone, and F. Scarcello. Hypertree Decompositions: Questions and Answers. In *Proc. of PODS'16*, pages 57–74, 2016.
- [18] G. Gottlob, C. Okulmus, and R. Pichler. Fast and parallel decomposition of constraint satisfaction problems. In C. Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pages 1155–1162. ijcai.org, 2020.
- [19] J. A. Grochow and M. Kellis. Network motif discovery using subgraph enumeration and symmetry-breaking. In *Research in Computational Molecular Biology, 11th Annual International Conference, RECOMB 2007, Oakland, CA, USA, April 21–25, 2007, Proceedings*, volume 4453 of *Lecture Notes in Computer Science*, pages 92–106. Springer, 2007.
- [20] M. Grohe and D. Marx. Constraint solving via fractional edge covers. *ACM Trans. Algorithms*, 11(1), aug 2014.
- [21] C. Gui, X. Liao, L. Zheng, and H. Jin. Cyclosa: Redundancy-free graph pattern mining via set dataflow. In *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10–12, 2023*, pages 71–85. USENIX Association, 2023.
- [22] W. Guo, Y. Li, M. Sha, B. He, X. Xiao, and K. Tan. Gpu-accelerated subgraph enumeration on partitioned graphs. In *SIGMOD*, pages 1067–1082. ACM, 2020.
- [23] M. Han, H. Kim, G. Gu, K. Park, and W.-S. Han. Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In *SIGMOD*, pages 1429–1446, 2019.
- [24] W.-S. Han, J. Lee, and J.-H. Lee. Turboiso: Towards Ultrafast and Robust Subgraph Isomorphism Search in Large Graph Databases. In *SIGMOD*, pages 337–348, 2013.
- [25] Z. He and J. X. Yu. A branch-&-bound algorithm for fractional hypertree decomposition. *Proc. VLDB Endow.*, 17(13):4655–4667, 2024.
- [26] P. Heggernes. Minimal triangulations of graphs: A survey. *Discret. Math.*, 306(3):297–317, 2006.
- [27] L. Hu, Y. Lin, L. Zou, and M. T. Özsu. A graph pattern mining framework for large graphs on GPU. *VLDB J.*, 34(1):6, 2025.
- [28] K. Jamshidi, R. Mahadasa, and K. Vora. Peregrine: a pattern-aware graph mining system. In *EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27–30, 2020*, pages 13:1–13:16. ACM, 2020.
- [29] X. Jian, Z. Li, and L. Chen. Suff: Accelerating subgraph matching with historical data. *Proc. VLDB Endow.*, 16(7):1699–1711, 2023.
- [30] H. Jiang, S. Pandey, and H. Liu. A comprehensive survey of subgraph matching-[experiments & analysis]. *Proceedings of the ACM on Management of Data*, 3(6):1–30, 2025.
- [31] T. Jin, B. Li, Y. Li, Q. Zhou, Q. Ma, Y. Zhao, H. Chen, and J. Cheng. Circinus: Fast redundancy-reduced subgraph matching. *Proceedings of the ACM on Management of Data*, 1(1):1–26, 2023.
- [32] O. Kalinsky, Y. Etsion, and B. Kimelfeld. Flexible caching in trie joins. In V. Markl, S. Orlando, B. Mitschang, P. Andritsos, K. Sattler, and S. Breß, editors, *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21–24, 2017*, pages 282–293. OpenProceedings.org, 2017.
- [33] H. Kim, Y. Choi, K. Park, X. Lin, S. Hong, and W. Han. Versatile equivalences: Speeding up subgraph query processing and subgraph matching. In *SIGMOD*, pages 925–937. ACM, 2021.
- [34] H. Kim, Y. Choi, K. Park, X. Lin, S.-H. Hong, and W.-S. Han. Fast subgraph query processing and subgraph matching via static and dynamic equivalences. *The VLDB journal*, 32(2):343–368, 2023.
- [35] H. Kim, J. Lee, S. S. Bhowmick, W. Han, J. Lee, S. Ko, and M. H. A. Jarrah. DUALSIM: parallel subgraph enumeration in a massive graph on a single machine. In *SIGMOD*, pages 1231–1245. ACM, 2016.
- [36] L. Lai, L. Qin, X. Lin, and L. Chang. Scalable subgraph enumeration in mapreduce: a cost-oriented approach. *VLDB J.*, 26(3):421–446, 2017.
- [37] L. Lai, L. Qin, X. Lin, Y. Zhang, and L. Chang. Scalable distributed subgraph enumeration. *Proc. VLDB Endow.*, 10(3):217–228, 2016.
- [38] A. Y. Levy, I. S. Mumick, and Y. Sagiv. Query optimization by predicate move-around. In *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, page 96–107, San Francisco, CA, USA, 1994. Morgan Kaufmann Publishers Inc.
- [39] Q. Li, J. Yu, and Z. He. Subgraph matching: A new decomposition based approach. *Proc. VLDB Endow.*, 18(11):4282–4294, 2025.
- [40] Q. Li and J. X. Yu. Fast local subgraph counting. *Proc. VLDB Endow.*, 17(8):1967–1980, 2024.
- [41] Z. Lin, K. Meng, C. Shui, K. Zhang, J. Xiao, and G. Tan. Exploiting fine-grained redundancy in set-centric graph pattern mining. In M. Steuwer, I. A. Lee, and M. Chabbi, editors, *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2024, Edinburgh, United Kingdom, March 2–6, 2024*, pages 175–187. ACM, 2024.
- [42] Y. Lu, Z. Zhang, and W. Zheng. B²X: Subgraph matching with batch backtrack-search. *Proc. ACM Manag. Data*, 3(1), Feb. 2025.
- [43] Y. Lu, Z. Zhang, W. Zheng, and L. Zou. Accelerating subgraph matching through fine-grained and powerful equivalences. *Proc. VLDB Endow.*, 18(11):3896–3909, 2025.
- [44] D. Marx and M. Pilipczuk. Everything you always wanted to know about the parameterized complexity of subgraph isomorphism (but were afraid to ask). In *31st International Symposium on Theoretical Aspects of Computer Science (STACS 2014), STACS 2014, March 5–8, 2014, Lyon, France*, volume 25 of *LIPIcs*, pages 542–553, 2014.
- [45] D. Mawhirter, S. Reinehr, C. Holmes, T. Liu, and B. Wu. Graphzero: Breaking symmetry for efficient graph mining. *CoRR*, abs/1911.12877, 2019.
- [46] A. Mhedhbi, A. Deshpande, and S. Salihoglu. Modern techniques for querying graph-structured databases. *Found. Trends Databases*, 14(2):72–185, 2024.
- [47] A. Mhedhbi and S. Salihoglu. Optimizing subgraph queries by combining binary and worst-case optimal joins. *Proc. VLDB Endow.*, 12(11):1692–1704, 2019.
- [48] H. Q. Ngo, E. Porat, C. Ré, and A. Rudra. Worst-case Optimal Join Algorithms: [Extended Abstract]. In *Proc. of PODS'12*, pages 37–48, 2012.
- [49] T. Ohtsuki, L. K. Cheung, and T. Fujisawa. Minimal triangulation of a graph and optimal pivoting order in a sparse matrix. *Journal of Mathematical Analysis and Applications*, 54(3):622–633, 1976.
- [50] M. Qiao, H. Zhang, and H. Cheng. Subgraph Matching: On Compression and Computation. *Proc. VLDB Endow.*, 11(2):176–188, 2017.
- [51] X. Qiu, W. Cen, Z. Qian, Y. Peng, Y. Zhang, X. Lin, and J. Zhou. Real-time constrained cycle detection in large dynamic graphs. *Proc. VLDB Endow.*, 11(12):1876–1888, 2018.
- [52] X. Ren, J. Wang, W. Han, and J. X. Yu. Fast and robust distributed subgraph enumeration. *Proc. VLDB Endow.*, 12(11):1344–1356, 2019.
- [53] R. A. Rossi and N. K. Ahmed. The network data repository with interactive graph analytics and visualization. In *AAAI*, 2015.
- [54] H. Shang, Y. Zhang, X. Lin, and J. X. Yu. Taming Verification Hardness: An Efficient Algorithm for Testing Subgraph Isomorphism. *Proc. VLDB Endow.*, 1(1):364–375, 2008.

- [55] N. Shervashidze, S. V. N. Vishwanathan, T. Petri, K. Mehlhorn, and K. M. Borgwardt. Efficient graphlet kernels for large graph comparison. In D. A. V. Dyk and M. Welling, editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics, AISTATS 2009, Clearwater Beach, Florida, USA, April 16-18, 2009*, volume 5 of *JMLR Proceedings*, pages 488–495. JMLR.org, 2009.
- [56] T. Shi, J. Zhai, H. Wang, Q. Chen, M. Zhai, Z. Hao, H. Yang, and W. Chen. Graphset: High performance graph mining through equivalent set transformations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2023, Denver, CO, USA, November 12-17, 2023*, pages 32:1–32:14. ACM, 2023.
- [57] T. Shi, M. Zhai, Y. Xu, and J. Zhai. Graphpi: high performance graph pattern matching through effective redundancy elimination. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 100. IEEE/ACM, 2020.
- [58] W. Shin, S. Song, K. Park, and W. Han. Cardinality estimation of subgraph matching: A filtering-sampling approach. *Proc. VLDB Endow.*, 17(7):1697–1709, 2024.
- [59] M. Y. Siraichi, V. F. dos Santos, C. Collange, and F. M. Q. Pereira. Qubit allocation as a combination of subgraph isomorphism and token swapping. *Proc. ACM Program. Lang.*, 3(OOPSLA):120:1–120:29, 2019.
- [60] S. Sun, Y. Che, L. Wang, and Q. Luo. Efficient parallel subgraph enumeration on a single machine. In *ICDE*, pages 232–243. IEEE, 2019.
- [61] S. Sun and Q. Luo. In-memory subgraph matching: An in-depth study. In *SIGMOD*, pages 1083–1098. ACM, 2020.
- [62] S. Sun, X. Sun, Y. Che, Q. Luo, and B. He. Rapidmatch: A holistic approach to subgraph query processing. *Proc. VLDB Endow.*, 14(2):176–188, 2020.
- [63] Z. Sun, H. Wang, H. Wang, B. Shao, and J. Li. Efficient subgraph matching on billion node graphs. *Proc. VLDB Endow.*, 5(9):788–799, 2012.
- [64] C. H. C. Teixeira, A. J. Fonseca, M. Serafini, G. Siganos, M. J. Zaki, and A. Aboul-naga. Arabesque: a system for distributed graph mining. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, pages 425–440. ACM, 2015.
- [65] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1):31–42, 1976.
- [66] T. L. Veldhuizen. Leapfrog triejoin: a worst-case optimal join algorithm. *CoRR*, abs/1210.0481, 2012.
- [67] Q. Wang and K. Yi. Conjunctive queries with comparisons. *SIGMOD Rec.*, 52(1):54–62, 2023.
- [68] Z. Wang, R. Gu, W. Hu, C. Yuan, and Y. Huang. BENU: distributed subgraph enumeration with backtracking-based framework. In *ICDE*, pages 136–147. IEEE, 2019.
- [69] S. Wernicke and F. Rasche. FANMOD: a tool for fast network motif detection. *Bioinform.*, 22(9):1152–1153, 2006.
- [70] D. E. Willard. Efficient processing of relational calculus expressions using range query theory. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data, SIGMOD '84*, page 164–175, New York, NY, USA, 1984. Association for Computing Machinery.
- [71] J. Yang and J. Leskovec. Defining and evaluating network communities based on ground-truth. pages 745–754. IEEE Computer Society, 2012.
- [72] Z. Yang, L. Lai, X. Lin, K. Hao, and W. Zhang. HUGE: an efficient and scalable subgraph enumeration system. In *SIGMOD*, pages 2049–2062. ACM, 2021.
- [73] M. Yannakakis. Algorithms for Acyclic Database Schemes. In *Very Large Data Bases, 7th International Conference, September 9-11, 1981, Cannes, France, Proceedings*, pages 82–94. IEEE Computer Society, 1981.
- [74] H. Zhang, M. Qiao, J. X. Yu, and H. Cheng. Fast distributed complex join processing. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pages 2087–2092. IEEE, 2021.
- [75] H. Zhang, J. X. Yu, Y. Zhang, K. Zhao, and H. Cheng. Distributed subgraph counting: A general approach. *Proc. VLDB Endow.*, 13(11):2493–2507, 2020.
- [76] Y. Zhang, W. Zheng, Z. Zhang, P. Peng, and X. Zhang. Hybrid subgraph matching framework powered by sketch tree for distributed systems. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pages 1031–1043. IEEE, 2022.
- [77] Z. Zhang, Y. Lu, W. Zheng, and X. Lin. A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction. *Proc. ACM Manag. Data*, 2(1):60:1–60:29, 2024.
- [78] Z. Zhang and W. Zheng. Bee: Towards redundancy reduction via block-separator decomposition for subgraph matching. *Proc. ACM Manag. Data*, 3(4), Sept. 2025.
- [79] P. Zhao and J. Han. On Graph Query Optimization in Large Networks. *Proc. VLDB Endow.*, 3(1-2):340–351, 2010.

Appendix

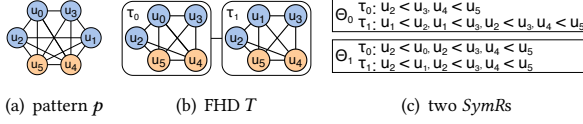


Figure 13: Different distributions of *SymRs*

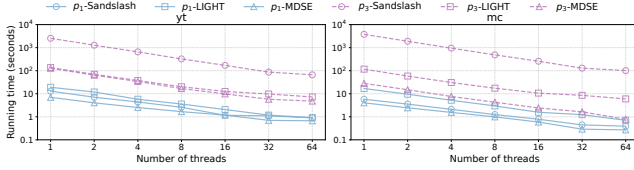


Figure 14: Scalability testing

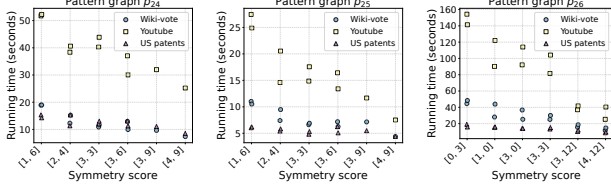


Figure 15: Effect of *SymRs* selection

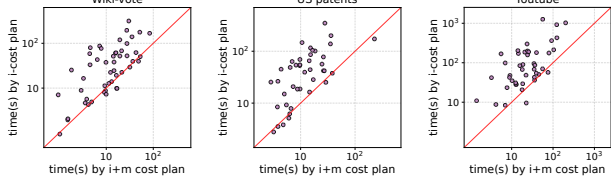


Figure 16: Effect of using materialization costs

A PROOFS

Proof of Lemma 4.1. We prove the lemma based on the fact that the fractional edge cover value is monotone, i.e., for any graph $G = (V, E)$, $S \subseteq V$ and $S' \subseteq S$, if $\rho(G(S))$ is the fractional edge cover value of the subgraph of G induced by S , and $\rho'(G(S'))$ is the value of S' , then $\rho'(G(S')) \leq \rho(G(S))$ [8]. Suppose for the contradiction that there does not exist such a MinFHD . Since $\text{fhtw}(p)$ is the minimum fractional hypertree width, this means that for any FHD T whose width is $\text{fhtw}(p)$, there exists another FHD (T', ρ') and a function $f : V_T \mapsto V_{T'}$, where $f(\tau_i) \subseteq \tau_i$ for all $\tau_i \in V_T$, $\exists \tau_j \in V_T$ such that $f(\tau_j) \subsetneq \tau_j$, and $\exists \tau'_k \in T'$ such that $\rho'(\tau'_k) > \text{fhtw}(p)$. However, by the monotonicity of fractional edge cover, for any $\tau'_i \in T'$, we have $\rho'(\tau'_i) \leq \rho(f^{-1}(\tau'_i)) \leq \text{fhtw}(p)$, which contradicts the existence of τ'_k with $\rho'(\tau'_k) > \text{fhtw}(p)$. $\square$

Proof of Lemma 4.2. First, if p_T^+ is a minimal triangulation but (T, ρ) is not a MinFHD , then there exists another FHD (T', ρ') and a function $f : V_T \mapsto V_{T'}$, where $\forall \tau_i \in T, f(\tau_i) \subseteq \tau_i$, and $\exists \tau_j \in V_T, f(\tau_j) \subsetneq \tau_j$. p_T^+ is a triangulation of p . By Definition 4.1 and the formation of p_T^+ and p_T^+ , we have $E_{p_T^+} \subsetneq E_{p_T^+}$, contradicting that p_T^+ is a minimal triangulation. Second, if (T, ρ) is a MinFHD but p_T^+ is not a minimal triangulation, then there exists a minimal

triangulation p^{++} such that $E_{p^{++}} \subsetneq E_{p_T^+}$. Then each maximal clique in p^{++} equals or is contained in a maximal clique of p_T^+ , and there is a maximal clique in p^{++} that is contained in a maximal clique in p_T^+ . We can construct a decomposition T' from the clique tree of p^{++} where T is not minimal and T' is minimal by Definition 4.1, a contradiction. $\square$

B DISCUSSIONS ON BALANCED COMPUTATION REDUCTION

Fig. 13 illustrates the effect of balanced predicate distribution across bags. We use a 6-node pattern graph p in Fig. 13(a) and its FHD T in Fig. 13(b). Symmetric nodes are filled with the same color (blue or orange). The two bags τ_0 and τ_1 in T induce isomorphic subgraphs, so they are in the same group in Algorithm 3. We compare two *SymRs* Θ_0 and Θ_1 as shown in Fig. 13(c), both using 6 rules. Θ_0 applies 2 rules in τ_0 and 4 rules in τ_1 , while Θ_1 applies 3 rules in both bags. On the wiki-vote data graph, Θ_0 takes 2.43 seconds while Θ_1 takes 1.75 seconds. This difference can be attributed to the sizes of bag relations. For Θ_0 , $R(\tau_0)$ contains 295M tuples and $R(\tau_1)$ contains 109M tuples. In contrast, for Θ_1 , both $R(\tau_0)$ and $R(\tau_1)$ contain 186M tuples, achieving more balanced computation for $R(\tau_0) \bowtie R(\tau_1)$. This motivates our use of multiplication instead of addition for the scoring metric, which favors balanced predicate distribution and better efficiency.

C ADDITIONAL EXPERIMENTS

Scalability testing. Fig. 14 compares thread scaling performance of Sandlash, LIGHT, and MDSE across patterns p_1 and p_3 on *yt* and *mc* datasets. Different line styles represent patterns p_1 and p_3 , and different point styles represent the three algorithms. All algorithms exhibit good scalability with execution times decreasing as thread count increases. MDSE consistently outperforms both baselines across all thread counts, demonstrating superior efficiency. MDSE employs straightforward pre-partitioning that divides workload into independent tasks for each thread, while LIGHT use sophisticated strategies including work-stealing and dynamic task redistribution. Despite its simpler parallelization approach, our tree decomposition-based method maintains good parallelizability and achieves reasonable speedups, indicating that the algorithmic efficiency of MDSE translates to parallel execution efficiency.

Symmetry score impact. Fig. 15 shows the correlation between symmetry scores and execution times across three patterns p_{24} , p_{25} , p_{26} in Fig. 8 and three data graphs *wv*, *yt*, *up*. Each plot displays execution times for different *SymRs* ranked by their symmetry scores. The results demonstrate substantial performance variations: pattern p_{24} shows 2.57 $\times$ difference between best/worst choices on *wv* (with 2.07 $\times$ on *yt*, 1.80 $\times$ on *up*), while pattern p_{26} exhibits the most dramatic 6.11 $\times$ variation on *yt*. The consistent downward trend from left to right confirms that higher scores correlate with better performance, validating our symmetry selection strategy.

Materialization cost model details. Fig. 16 compares intersection-only cost modeling versus comprehensive modeling that includes both intersection and materialization costs. Each scatter plot shows

Table 5: Ablation study (time in seconds)

Dataset	BASE	+MFHD	+MFHD+SYM	+MFHD+SYM+MIX	ALL
<i>wv</i>	9.47	4.74	3.96	3.62	2.24
<i>up</i>	7.43	7.07	5.75	4.79	3.43
<i>yt</i>	483.22	32.55	26.93	20.91	14.41
<i>mc</i>	4088.75	4159.62	3491.21	1694.88	1157.87

execution times for patterns under both cost models, omitting cases where both models produce identical plans. The comprehensive approach achieves faster or equivalent execution in 83.7% of queries on *wv*, 85.4% on *up*, and 95.5% on *yt*, with average speedups of 3.8 $\times$, 4.2 $\times$, and 5.6 $\times$ respectively. The results validates the superiority of our cost model since it includes the materialization costs.

Ablation Study. Table 5 evaluates the incremental contribution of each MDSE component, reporting average execution time per pattern across 16 6-node patterns on four datasets. We impose a 2-hour timeout. Instances exceeding the limit are recorded as 7200 seconds. We compare five configurations: BASE uses the FHD

selection from [25] with SCOPE’s symmetry breaking [40] and ASDJoin [39]; +MFHD replaces the FHD selection with our MinFHD approach; +MFHD+SYM additionally adopts our symmetry selection; +MFHD+SYM+MIX further replaces ASDJoin with MixJoin using the intersection-only cost model; ALL is the complete MDSE system with the materialization-aware cost model. On *mc*, all configurations except ALL have at least one timeout, and on *yt*, BASE has one timeout, so the actual speedups would be even larger. Each component contributes to the overall performance. MinFHD selection yields up to 1.05 $\times$ –14.8 $\times$ speedup and most notably on *yt* where the baseline often selects suboptimal decompositions, causing some instances to time out. Symmetry selection provides consistent 1.2 $\times$ improvements across all datasets. MixJoin achieves 1.1 $\times$ –2.1 $\times$ speedup and is most impactful on *mc* where large intermediate results make the extra materialization step in ASDJoin costly. The materialization-aware cost model further improves all four datasets by 1.4 $\times$ –1.6 $\times$. Overall, the complete system achieves 2.2 $\times$ –33.5 $\times$ total speedup over BASE.